C++ Embedded Interview Technical Notes

By Renato Soriano De la Rosa

http://linkedin.com/in/renatosoriano

http://github.com/renatosoriano

Table of Contents

Constructor and Destructor

Objects are instances of classes. They are variables that occupy memory. Special functions, called constructors, are used to construct or instantiate objects.

Constructors are used to initialize objects, including the initialization of class members, and destructors are used to clean up resources. An object is created using a constructor, and when the object variable goes out of scope, the destructor is called.

Constructors and destructors both increase the size of the binary and add runtime overhead, as their execution takes time.

Example of a class with one private member, a constructor, a destructor, and a getter:

```
class MyClass
{
    private:
        int num;
    public:
        MyClass(int t_num):num(t_num){}
        ~MyClass(){}

        int getNum() const {
            return num;
        }
};

int main () {
    MyClass obj(1);
    return obj.getNum();
}
```

As of C++11, it is possible to set a default value for a member directly in a class definition, as follows:

```
class my_class{
    int a = 4;
    int *ptr = nullptr;
}
```

The default member initializers allow us to set a default value for class members in a class definition, which improves readability and saves us from setting the same member variable if we have multiple constructors. This is particularly useful for setting default values for pointers.
If we didn't use the default initializer for ptr, it would be loaded with some random value from memory. Dereferencing such a pointer would result in reading from or writing to a random location, potentially leading to a serious fault.

Constructors and member initializer lists

Constructors are nameless methods in class definition that can't be called explicitly. They are invoked upon the object initialization. A constructor that can be invoked with no arguments is called the default constructor:

```
#include <cstdint>
#include <cstdio>

class uart {

public:
    uart(std::uint32_t baud = 9600): baudrate_(baud) {
        printf("%s\n", __func__);
    }

private:
    std::uint32_t baudrate_;
};

int main () {
    uart uart1(115200);
    return 0;
}
```

We used the default argument that will be provided to the constructor if it is called with no arguments. If no argument is provided at the call site, the default value of 9600 will be used for the baud argument.

When we want to use the default constructor:

```
uart uart1;
```

This is also called default initialization, and it is performed when the object is declared with no initializer.

As uart class constructor can also accept an argument, we can use direct initialization syntax and provide the constructor with an argument:

```
uart uart1(115200);
```

This will call the uart class constructor and provide it with a value of 115200 for the baud argument.

For the initialization of the baudrate_ member variable, we are using the member initializer list. It is specified after the colon character and before the opening brace of the compound statement as baudrate_(baud).

If a constructor has one parameter and is declared without the explicit specifier, it is called a converting constructor.

Converting constructors and explicit specifiers

Converting constructors allow the compiler to make an implicit conversion from the type of its argument to the type of its class:

```
#include <cstdio>

struct uart {
    uart(std::uint32_t baud = 9600): baudrate_(baud) {}
    std::uint32_t baudrate_;
};

void uart_consumer(uart u) {
    printf("Uart baudrate is %d\r\n", u.baudrate_);
}
```

```
int main() {
    uart uart1;
    uart_consumer(uart1);
    uart_consumer(115200);
    return 0;
}
```

The interesting part of this example is the call to the uart_consumer function with the 115200 argument. The uart_consumer function expects the object of the uart class as an argument, but due to rules of implicit conversion and the existing converting constructor, the compiler constructs an object of the uart class using 115200 as an argument, resulting in the following output of the program:

```
Uart baudrate is 9600
Uart baudrate is 115200
```

Implicit conversion can be unsafe, and it is often unwanted. To prevent it, we can declare a constructor using an explicit specifier, as follows:

```
explicit uart(std::uint32_t baud = 9600): baudrate_(baud) {}
```

Compiling the preceding example with an explicit constructor will result in a compiler error:

```
<source>:19:19: error: could not convert '115200' from 'int' to 'uart'
19 | uart_consumer(115200);
```

By declaring a constructor as explicit, we can be sure that no user of our class will create a situation with potential implicit conversion, which may lead to unwanted behavior in our program.

We can delete specific constructors, which will result in failed compilation. We can do it using the following syntax in the class declaration:

```
uart(float) = delete;
```

Destructors

A destructor is a special member function of a class that is automatically called when an object goes out of scope or is explicitly deleted.

Syntax:

```
~ClassName() { /* cleanup */ }
```

Purpose: release resources (memory, file handles, hardware peripherals, etc).

Key points:

- You can only have one destructor per class (no overloading).
- Called in reverse order of construction (last constructed, first destroyed).
- If not declared, the compiler generates a default destructor (does nothing special).

Virtual Destructor

A destructor should be virtual if your class is intended to be used polymorphically (i.e., through a base-class pointer or reference). If you delete a derived object through a base pointer without a virtual destructor, only the base's destructor runs → derived resources leak.

Example: Non-virtual Destructor (BAD ❌)

```
#include <iostream>

class Base {
public:
    ~Base() { std::cout << "Base destructor\n"; }
};

class Derived : public Base {
public:
    ~Derived() { std::cout << "Derived destructor\n"; }
};

int main() {
    Base* obj = new Derived();
    delete obj;  // ⚠ Only Base::~Base runs, Derived::~Derived skipped!
}
```

Output:

```
Base destructor
```

⚠ Leak: any resources in Derived are not freed.

Example: Virtual Destructor (GOOD ✅):

```cpp
#include <iostream>

class Base {
public:
    virtual ~Base() { std::cout << "Base destructor\n"; }
};

class Derived : public Base {
public:
    ~Derived() override { std::cout << "Derived destructor\n"; }
};

int main() {
    Base* obj = new Derived();
    delete obj;  // Both destructors run
}
```

Output:

```
Derived destructor
Base destructor
```

✓ Proper cleanup, no leaks.

Performance Considerations:

- Virtual destructors add one vtable pointer per polymorphic class → tiny memory/time cost.
- If a class is not intended to be inherited from polymorphically (e.g., utility classes, POD-like structs), don't make the destructor virtual.
- Use virtual destructors only where needed.

Best Practices:

1. Base classes for polymorphism:

```cpp
virtual ~Base() = default;
```

(even better, make it pure virtual if it's an abstract base).

2. Non-polymorphic classes → default non-virtual destructor.
3. Final classes (final in C++11) → no need for virtual destructor since they can't be derived from.
4. If a class manages resources (heap, hardware, file, mutex), destructor is a natural place to clean them up (RAII principle).

Embedded C++ Context:

- RAII + deterministic cleanup: Important for releasing MCU peripherals (GPIO, UART, SPI, DMA, etc.) automatically when objects go out of scope.
- Virtual destructors:
  - ✅ Use in driver frameworks where a base interface (e.g., ICommInterface) may point to different hardware backends (UartDriver, SpiDriver).
  - ❌ Avoid in tiny leaf classes where polymorphism isn't needed — save flash/RAM (vtable adds overhead).
- Alternatives: In very constrained environments, you might avoid polymorphism altogether and use templates (CRTP) or static polymorphism for zero overhead.

Rule of Thumb:
If you'll ever delete a derived object through a base pointer → make the base destructor virtual.
If not, leave it non-virtual for efficiency.

🔷 Summary Table

Case	Destructor Virtual?	Why
Class is **not** used polymorphically	❌ No	Saves space, faster, no vtable
Class is a **base class** with virtual methods	✅ Yes	Ensures derived destructors run
Class is an **abstract interface**	✅ Yes	Always delete via base pointer safely
Class is `final` (cannot be derived)	❌ No	No need for polymorphism
Embedded bare-metal class managing hardware	Usually ❌	Use RAII, but polymorphism often avoided for determinism

Classes – OOP Building Blocks

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects"—data structures that contain both data (attributes) and functions (methods) that operate on the data.

Classes in C++ are means of organizing code into logical units. They allow us to structure data and functions that perform operations on that data in blueprints. These blueprints can be used to build instances of the classes, known as objects. We can initialize objects with data, manipulate them by calling functions or methods on them, store them in containers, or pass their references to objects of other classes to make the interaction between different parts of a system.

1. Encapsulation: Hiding the Internal State

Encapsulation is about **data hiding**. It protects the internal state of an object and only exposes necessary parts via public methods. The idea of encapsulation is to hide the implementation details, like data members.

```
class BankAccount {
private:
    double balance;

public:
    BankAccount(double initialBalance) {
        balance = initialBalance;
    }

    void deposit(double amount) {
        if (amount > 0) balance += amount;
    }

    double getBalance() {
        return balance;
    }
};
```

Why it matters: Reduces complexity and increases security by preventing outside interference and misuse.

C++ has the following specifiers:
- Public: Members are accessible from anywhere.
- Private: Members are only accessible within the class itself.

• Protected: Members are accessible within the class and by derived classes.

Publicly accessible members are the interface for our class. It is a common practice for data members that we want to expose to the public to define setter and getter methods.

The private access specifier allows us to hide methods and data from the user of our class. By controlling which methods a user of the class can use, we are not only protecting the class but also the user from unwanted behavior. The default access specifier for classes in C++ is private.

Static methods

Static methods are C++ methods declared with static keywords, and they are accessible without object instantiation.

```
#include <cstdint>

class uart {
public:
    static void init(std::uint32_t baudrate) {
        write_brr(calculate_uartdiv(baudrate));
    }
private:
    static std::uint8_t calculate_uartdiv(std::uint32_t baudrate) {
        return baudrate / 32000;
    }
    static void write_brr(std::uint8_t) {}
};

int main () {
    uart::init(115200);
    return 0;
}
```

Static methods can only use static data members and other static functions from a class as non-static members require the instantiation of an object.

Besides the public and private access specifiers, there is also the protected specifier in C++. The protected specifier is related to inheritance.

2. Abstraction: Focusing on What, Not How

Abstraction allows you to hide complex implementation details and show only the **essential features** of the object through a clear interface.

```
class Vehicle {
public:
    virtual void startEngine() = 0; // Pure virtual function
};
```

Why it matters: Simplifies the interface for the user, encouraging clean and efficient design. Provides flexibility to change internal implementations without affecting external code.

Virtual Functions

Virtual functions in C++ are usually implemented using virtual tables. This is the work that a compiler does for us. It creates a virtual table that stores pointers for every virtual function, which points to the overridden implementation.

Every object of a class has a pointer to this table. This pointer is used at runtime to access a table and find the correct function to be called on the object.

Example of code for class A and class B:

```
class A {
public:
    virtual void method_1() {
        printf("Class A, method1\r\n");
    }
    virtual void method_2() {
        printf("Class A, method2\r\n");
    }
};
class B : public A{
public:
    void method_2() override {
        printf("Class B, method2\r\n");
    }
};
```

class A has two virtual methods, method_1 and method_2. class B only overrides method_2. If we call method_2 on a reference to an object of class B, it will follow the virtual pointer to the virtual table and select the function pointer that points to the implementation of method_2 in class B, that is, the overridden virtual function.

The **override** keyword makes the compiler aware of our intention of overriding a virtual method from the base class. If the method we are overriding is not declared virtual, the compiler will raise an error.

Pure Virtual Functions

In C++, we can define a virtual function to be a pure virtual function. If a class has a pure virtual function, it is called **an abstract class**, and it can't be instantiated. Derived classes must **override** pure virtual functions, or they are also abstract classes.

A pure virtual function is a function in a base class that has no definition. It is declared using = 0 at the end of the function declaration. A class containing a pure virtual function is an abstract class and cannot be instantiated. Derived classes must implement this function.

```
#include <iostream>
using namespace std;

// Abstract Class
class Shape {
public:
 virtual void draw() = 0; // pure virtual function
 virtual double area() = 0; // pure virtual function
 virtual ~Shape() {} // virtual destructor
};

// Derived Class: Circle
class Circle : public Shape {
 double radius;
public:
 Circle(double r) : radius(r) {}
 void draw() override {
            cout << "Drawing Circle\n";
 }
 double area() override {
            return 3.14159 * radius * radius;
```

```
 }
};

// Derived Class: Rectangle
class Rectangle : public Shape {
 double width, height;
public:
 Rectangle(double w, double h) : width(w), height(h) {}
 void draw() override {
            cout << "Drawing Rectangle\n";
 }
 double area() override {
            return width * height;
 }
};

int main() {
 Shape* s1 = new Circle(5);
 Shape* s2 = new Rectangle(4, 6);

 s1->draw();
 cout << "Area: " << s1->area() << endl;

 s2->draw();
 cout << "Area: " << s2->area() << endl;

 delete s1;
 delete s2;
}
```

Shape (abstract/interface class) defines what operations exist. Example: draw(), area() -> two pure virtual functions.

Interface is inherited and implemented by:

Circle & Rectangle (derived classes) define how these operations are performed. Example: Circle uses πr^2 for area, Rectangle uses width × height.

This is abstraction → users only care about the interface (Shape), not the internal details.
Easily extend the program with new shapes without changing the existing code logic.

3. Inheritance: Reusability at Its Best

Inheritance is an example of establishing a hierarchical relationship (**inherit properties and behavior**) between classes.

```
#include <cstdio>

class A {
public:
    void method_1() {
        printf("Class A, method1\r\n");
    }
    void method_2() {
        printf("Class A, method2\r\n");
    }
protected:
    void method_protected() {
        printf("Class A, method_protected\r\n");
    }
};

class B : public A{
public:
    void method_1() {
        printf("Class B, method1\r\n");
    }
    void method_3() {
        printf("Class B, method3\r\n");
        A::method_2();
        A::method_protected();
    }
};
```

```
int main() {
    B b;
    b.method_1();
    b.method_2();
    b.method_3();
    printf("----------------\r\n");
    A a;
    a.method_1();
    a.method_2();
    return 0;
}
```

class B inherits public and protected members from class A. class A is the base class, and class B is derived from it. The derived class has access to public and protected members of the base class.

The output of object of class B code is shown here:
Class B, method1
Class A, method2
Class B, method3
Class A, method2
Class A, method_protected

Class A, method1
Class A, method2

The call to the method_1 function on object b executes method_1 defined in class B even though it is derived from class A, is called static binding.

Why it matters: Promotes code reuse, reduces redundancy, and supports hierarchical classification.

4. Polymorphism: One Interface, Multiple Forms

Polymorphism allows the same function or method to behave **differently depending on the object**.

4.1 Static: Compile-time Polymorphism

- Resolved at compile time by the compiler.
- Also known as: Static binding or early binding.
- Performance: Faster (no runtime overhead).

- Flexibility: Less flexible, behavior fixed at compile time.
- Achieved using:
    - Function overloading
    - Operator overloading
    - Templates (parametric polymorphism)

The compiler decides which function/operator to call based on argument matching.

```cpp
#include <iostream>
using namespace std;

class Math {
public:
    // Function overloading
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }

    // Operator overloading
    int operator()(int a, int b) {
        return a * b; // acts like multiplication function
    }
};

int main() {
    Math m;
    cout << m.add(3, 4) << endl;     // calls int add
    cout << m.add(3.5, 2.5) << endl; // calls double add
    cout << m.add(1, 2, 3) << endl;  // calls 3-parameter add
    cout << m(5, 6) << endl;         // operator() overloading
}
```

Compile-time polymorphism → function/operator overloading, templates.

Operator Overloading

Is a feature that allows you to redefine the way operators work for user-defined types (like classes). It enables you to give special meaning to an operator (e.g., +, -, *, etc.) when it is used

with objects of a class. The goal is to redefine the behavior of an operator so that it works with objects of a user-defined type, while still retaining its meaning for built-in types.

Syntax of Operator Overloading:

```
return_type operator op (arguments) {
    // function body
}
```

Example:

```
#include <iostream>
using namespace std;

class Complex {
private:
    float real, imag;

public:
    Complex(float r = 0, float i = 0) : real(r), imag(i) {}

    // Overload the '+' operator
    Complex operator+(const Complex& other) {
        return Complex(real + other.real, imag + other.imag);
    }

    void display() const {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1(3.5, 2.5);
    Complex c2(1.5, 4.5);

    Complex result = c1 + c2;  // Operator '+' is overloaded
    cout << "Sum = ";
    result.display();
```

```
    return 0;
}
```

Output: Sum = 5 + 7i

Almost all operators can be overloaded except a few. Following is the list of operators that cannot be overloaded: sizeof, typeid, Scope resolution (::), Class member access (.(dot), .* (pointer to member operator)), Ternary or conditional (?:).

Important Points:

1. For operator overloading to work, at least one of the operands must be a user-defined class object.
2. Assignment Operator: Compiler automatically creates a default assignment operator with every class. The default assignment operator does assign all members of the right side to the left side and works fine in most cases.
3. Conversion Operator: We can also write conversion operators that can be used to convert one type to another type. Overloaded conversion operators must be a member method. Other operators can either be the member method or the global method.
4. Any constructor that can be called with a single argument works as a conversion constructor, which means it can also be used for implicit conversion to the class being constructed.

4.2 Dynamic: Runtime Polymorphism

- Resolved at runtime using the vtable mechanism.
- Also known as: Dynamic binding or late binding.
- Performance: Slightly slower due to virtual table lookup.
- Flexibility: More flexible, supports dynamic behavior.
- Achieved using:
    - Virtual functions in base classes.
    - Method overriding in derived classes.
    - Abstract classes / Pure virtual functions.

At runtime, the actual object type determines which function implementation runs, not the pointer/reference type.

```
#include <span>
#include <cstdio>
#include <cstdint>
```

```
class uart {
public:
    virtual void init(std::uint32_t baudrate) = 0;
    virtual void write(std::span<const char> data) = 0;
};

class uart_stm32 : public uart{
public:
    void init(std::uint32_t baudrate = 9600) override {
        printf("uart_stm32::init: setting baudrate to %d\r\n", baudrate);
    }
    void write(std::span<const char> data) override {
        printf("uart_stm32::write: ");
        for(auto ch: data) {
            putc(ch, stdout);
        }
    }
};

class gsm_lib{
    public:
        gsm_lib(uart &u) : uart_(u) {}
        void init() {
            printf("gsm_lib::init: sending AT command\r\n");
            uart_.write("AT");
        }
    private:
        uart &uart_;
};

int main() {
    uart_stm32 uart_stm32_obj;
    uart_stm32_obj.init(115200);
    gsm_lib gsm(uart_stm32_obj);
    gsm.init();
    return 0;
```

```
}
```

In this code example, we see that the uart class has two pure virtual functions, which makes it an interface class. This interface is inherited and implemented by the uart_stm32 class. In the main function, we create an object of the uart_stm32 class, whose reference is passed to the constructor of the gsm_lib class, where it is used to initialize a private member reference to the uart interface.

• Runtime polymorphism → virtual functions, overriding, abstract classes.

Fundamental Concepts

Namespaces

Namespaces: scope specifiers
Scope resolution operator (::)

In C++, we can use namespaces as scope specifiers for accessing type names, functions, variables, etc instead of C-style identifier prefixes to organize code in logical groups.

Is a declarative region that provides scope to identifiers like variables, functions, and classes. It's mainly used to organize code and avoid name conflicts, especially when integrating multiple libraries.

```
namespace hal {
    void init();

    std::uint32_t tick_count;
    std::uint32_t get_ms() {
        return tick_count;
    }
    class uart_stm32 {
    private:
        UART_HandleTypeDef huart_;
        USART_TypeDef *instance_;
    };
};
```

All members of the hal namespace are accessible unqualified from within the namespace.

```
hal::init();
std::uint32_t time_now = hal::get_ms();
```

***** C++ standard library types and functions are declared in the std namespace. ****

Using Directive

We can use the using directive to access an identifier without qualifiers:

```
using std::array;
array<int, 4> arr;
```

std = qualifiers
array = identifier

The using directive can also be used for the entire namespace:

```
using namespace std;
array<int, 4> arr;
vector<int> vec;
```

It is recommended to use using directive sparingly, especially with std, using it for a limited scope, or even better, to bring in individual identifiers only.

Functions and types that are not declared within an explicit namespace are part of a global namespace.

It is a good practice to organize all code in namespaces. The only function that can't be declared within a namespace and must be in a global namespace is main. To access the identifier from the global namespace, we use the scope resolution operator:

```
const int ret_val = 0;
int main() {
    return ::ret_val;
}
```

The line return ::ret_val; uses the scope resolution operator, ::, without specifying a namespace. This means it refers to the global namespace. So, ::ret_val accesses the ret_val variable defined outside of any function or class—that is, at the global scope.

constexpr: C++ equivalent to C macros

- Introduced in: C++11 (enhanced in C++14, C++17, C++20).

- What it does: Allows evaluation of functions/variables at compile time (if possible). This helps generate faster, safer code and can replace macros with type safety.

Code example:

```
// C++11 constexpr
constexpr int square(int x) { return x * x; }

int main() {
    int arr[square(4)];  // array of size 16, computed at compile-time
}
```

C++14+ relaxed rules → more complex bodies allowed.
C++20 added consteval (see below).

C++11 introduced the constexpr specifier, which declares that it is possible to evaluate the value of the function or a variable at compile time.

A constexpr variable must be immediately initialized, and its type must be a literal type (int, float, etc.). This is how we declare a constexpr variable:

```
constexpr double pi = 3.14159265359;
```

constexpr specifier is the preferred way of declaring compile-time constants in C++ over using a C-style approach with macros.

<u>In Embedded C++:</u>

✅ Best use: Precompute lookup tables, hardware register masks, pin mappings, or math constants at compile-time (saves CPU cycles).
⚠️ Don't abuse with huge constexpr tables → flash usage goes up.

```
constexpr uint32_t baud_reg(uint32_t sysclk, uint32_t baud) {
    return sysclk / baud;
}
```

```
constexpr auto baud115200 = baud_reg(16'000'000, 115200); // computed at compile time
```

Embedded benefit: UART init is faster, no runtime division needed.

consteval: evaluate at compile

- Introduced in: C++20.

- What it does: Forces a function to be evaluated at compile time only. Unlike constexpr, you cannot call it at runtime.

Code example:

```
consteval int cube(int x) {
    return x * x * x;
}

int main() {
    constexpr int val = cube(3); // ✅ ok, computed at compile time
    // int n; cube(n); // ❌ error, must be compile-time
}
```

In Embedded C++:

✅ Best use: Enforce compile-time configuration (e.g., board pin layout).
⚠️ Cannot run at runtime, so it's only for constants.

```
consteval int check_pwm_range(int duty) {
    if (duty < 0 || duty > 1000) throw "Invalid duty"; // compile-time error
    return duty;
}

constexpr int motor_duty = check_pwm_range(500); // ✅ ok
// constexpr int bad = check_pwm_range(2000);    // ❌ compile-time error
```

Embedded benefit: Detect misconfigurations (pins, registers, duty cycles) before flashing firmware.

auto

- Introduced in: C++11 (type deduction).
- Improved in C++14 (return type deduction).
- Extended in C++20 (concepts + abbreviated function templates).

- What it does: Lets the compiler deduce the type of a variable or function return, reducing verbosity.

Code example:

```
auto x = 42;      // int
auto y = 3.14;    // double


auto add(auto a, auto b) { return a + b; } // C++20 abbreviated template
```

The auto keyword was introduced in C++11, and when used, the compiler deduces the actual type of a variable from the initialization expression.

In Embedded C++:

✅ Best use: Cleaner peripheral code, avoids mismatched types from CMSIS or HAL.
⚠️ Be explicit in public APIs; auto is great for internal code but can hide type width (uint16_t vs uint32_t).

```
auto val = ADC1->DR; // compiler deduces type (uint32_t)
```

Embedded benefit: Less boilerplate when handling HAL registers or iterators over buffers.

Lifetime and Scope of an Instance

In C++, the lifetime of an instance (object) is distinct from the scope of a variable that refers to it. Lifetime refers to the period during which an object exists and occupies memory, while scope refers to the region of code where a variable name is visible and accessible.

1. Value (Object by Value)

- Lifetime: The lifetime of an object created by value (e.g., a local variable) begins when its declaration is encountered and its initialization is complete. Its lifetime ends when it goes

out of scope, at which point its destructor is called (if it's a class type) and its memory is released.

- Scope: The scope of a value variable is typically limited to the block in which it is declared (e.g., a function, a loop, or a conditional block). Once the execution leaves that block, the variable is no longer accessible.

Example:

```
void myFunction() {
    int x = 10; // Lifetime and scope of 'x' begin here
    // ...
} // Lifetime and scope of 'x' end here
```

2. Reference

- Lifetime: A reference itself does not own an object; it is an alias for an existing object. Therefore, its lifetime is tied to the lifetime of the object it refers to. If the referred-to object is destroyed before the reference goes out of scope, the reference becomes invalid, leading to undefined behavior if accessed.

- Scope: The scope of a reference variable is determined by where it is declared, similar to a regular variable.

- Important Note: References cannot be reseated to refer to a different object after initialization.

Example:

```
int globalVar = 20;

void anotherFunction() {
    int y = 30;
    int& refY = y; // refY refers to y. Lifetime of refY is tied to y.
    int& refGlobal = globalVar; // refGlobal refers to globalVar.
    // ...
} // refY goes out of scope, but y still exists until anotherFunction ends.
  // refGlobal goes out of scope, but globalVar continues to exist.
```

3. Pointer

- Lifetime: A raw pointer variable itself has a lifetime determined by its scope, just like any other variable. However, the object it points to has its own independent lifetime.
    - If the pointer points to an object on the stack, the object's lifetime ends when its scope ends.
    - If the pointer points to dynamically allocated memory (using new), the object's lifetime continues until delete is explicitly called on that pointer. Failure to delete leads to memory leaks.
- Scope: The scope of a pointer variable is determined by where it is declared.
- Flexibility: Pointers can be reassigned to point to different objects during their lifetime. They can also point to nullptr (or NULL) to indicate that they are not currently pointing to any valid object.
- Smart Pointers: To manage the lifetime of dynamically allocated objects more safely, C++ offers smart pointers like std::unique_ptr, std::shared_ptr, and std::weak_ptr, which automatically handle memory deallocation.

Example:

```
void yetAnotherFunction() {
    int* ptrStack = nullptr;
    int z = 40;
    ptrStack = &z; // ptrStack points to z. Lifetime of z ends when yetAnotherFunction ends.

    int* ptrHeap = new int(50); // Dynamically allocated memory.
    // ...
    delete ptrHeap; // Explicitly deallocate memory.
    ptrHeap = nullptr; // Good practice to set to nullptr after deletion.
}
```

Standard Template Library (STL)

It is a powerful library that provides ready-to-use classes and functions for common data structures and algorithms, implemented using templates.

Four pillars of STL: containers, algorithms, iterators, functors.

Why Use STL?

- Efficiency: Optimized, tested, and reusable code.
- Productivity: Saves time writing common data structures and algorithms.
- Generic: Works with any data type because it's template-based.
- Flexibility: Easy to switch between different container types.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> nums = {5, 2, 9, 1, 7};

    // Sorting using STL algorithm
    sort(nums.begin(), nums.end());

    // Printing using iterator
    for (auto it = nums.begin(); it != nums.end(); ++it) {
        cout << *it << " ";
    }
    return 0;
}
```

Container → vector<int>
Algorithm → sort()
Iterators → nums.begin(), nums.end()
(No functor in this example, but we could use one like greater<int>() for descending sort)

0. Headers

<cstdint>

The <cstdint> header provides fixed-width integer types such as std::int8_t, std::uint32_t, and other well-known C types defined in stdint.h. These types are useful in embedded contexts where integer size and bit width are important for direct hardware register access, predictable overflow behavior, and memory usage considerations. By using them, you explicitly document your

intention for a variable's size, thereby improving code portability and preventing potential surprises when moving between platforms with different native integer widths.

<limits>

The header provides the std::numeric_limits template, which describes properties of fundamental numeric types (like minimum and maximum values, sign, and precision). This is especially useful in embedded contexts for handling overflow. Typical usage occurs at compile-time or through trivial inlining by the compiler, resulting in minimal runtime overhead.

<cmath>

The <cmath> header provides standard math functions such as std::sin, std::cos, std::sqrt, and more. In embedded environments, especially those without floating-point hardware, these functions can be relatively expensive in terms of both runtime performance and code size.

1. Containers: Store data

Sequence containers (ordered collection)

Vector (dynamic array)

Standard: C++98.
What: Dynamic array that grows as you push elements. Contiguous in memory → great cache locality; random access is O(1).

Can resize itself automatically as elements are added or removed. It provides fast random access and is typically used when the number of elements is not known in advance or changes frequently.

Ways to declare / define:

```
#include <vector>

// empty, then grow
std::vector<int> v1;
v1.push_back(10);

// with count and default value
std::vector<int> v2(5, 42);     // {42,42,42,42,42}
```

```cpp
// from initializer list
std::vector<int> v3{1,2,3,4};

// from iterators
int raw[] = {7,8,9};
std::vector<int> v4(std::begin(raw), std::end(raw));

// reserve to avoid reallocations
std::vector<int> v5;
v5.reserve(128);
```

Embedded tips:

- Uses the heap. On bare-metal, either:
  - pre-reserve() to avoid runtime reallocations,
  - replace with a fixed-capacity alternative (e.g., ETL vector, Boost.Container static_vector),
  - or use std::array if size is fixed.
- Good for DMA-able contiguous buffers (when alignment fits).

---

std::vector is a template class, and we can specify the underlying type. It stores the elements contiguously, and we can get direct access to the underlying contiguous storage using the data method. When the vector fills the available memory, it requests allocation of the doubled amount of currently used memory.

The allocation requests from the vector can be optimized by using the reserve method if the number of elements is known beforehand. This makes it useful for projects that allow dynamic allocation at startup if we can guarantee that the number of elements at any point will stay within reserved memory.

```cpp
std::vector<int> numbers;
vector<int> nums = {5, 2, 9, 1, 7};
vector<int> v1 {5,2}; -> {5, 2}
vector<int> v1 (5,2); -> {2, 2, 2, 2, 2}
```

You can substitute it with etl::vector from ETL, avoiding issues with dynamic memory allocation.

Array

Standard: C++11.

What: Fixed-size array with STL interface. Size is compile-time constant; no heap.

Ways to declare / define:

```
#include <array>

std::array<uint16_t, 64> adcBuf{};     // zero-initialized
auto &first = adcBuf[0];

constexpr std::array<int, 3> lut{{ 2, 4, 8 }}; // constexpr-friendly

// fill
adcBuf.fill(0xFFFF);
```

Embedded tips:

- Perfect for deterministic memory and ISR-safe storage.
- Works with all algorithms (via iterators).
- Prefer over C arrays for clearer intent and safety.

As std::vector relies on dynamic memory allocation, std::array is usually the container of choice in embedded applications.

```
std::array<int, 5> arr = {5, 3, 4, 1, 2};
std::array<int, 5>::iterator start = arr.begin();
```

The only fixed-size container in the standard library that avoids dynamic allocation is std::array. std::array is typically implemented as a wrapper around a C-style array. Besides being a generic type, it also offers the at method for index-based access with runtime bounds checking, making it a safer alternative to raw arrays. If an out-of-bounds index is requested, the at method will throw an exception. If exceptions are disabled, it may call std::terminate or std::abort, depending on the library implementation.

Arrays from the standard library allocate a contiguous block of memory on the stack. We can consider an array as a simple wrapper of a C-style array that contains the size of the array inside the type. It is a templated type that is instantiated with an underlying data type and size.

We can access members of the array using a method that will throw an exception if indexed with an out-of-bounds index. This makes it a safer option than a C-style array as it allows us to catch out-of-bounds access runtime errors and handle them.

We can use std:array to create a vector-like container that we can use with container adaptors such as std::stack or a std::priority queue.

Array-to-pointer conversion

An array can be implicitly converted to a pointer. The resulting pointer points to the first element of the array. Many C and C++ functions that work on arrays of data are designed with pointer and size parameters. These interfaces are based on contract design. The contract is the following:
• A caller will pass a pointer that points to the first element of the array.
• A caller will pass the size of the array.

```
#include <cstdio>

void print_ints(int * arr, std::size_t len) {
    for(std::size_t i = 0; i < len; i++) {
        printf("%d\r\n", arr[i]);
    }
}

int main() {
    int array_ints[3] = {1, 2, 3};
    print_ints(array_ints, 3);
    return 0;
}
```

In the above example, we have the print_ints function with arr, a pointer to an int, and len, a std::size_t parameter. In the main function, we call the print_ints function by passing array_ints, an array of 3 integers, and 3 as arguments. The array array_ints will be implicitly converted to a pointer that points to its first element. There are a couple of potential issues with the print_ints function:

- It expects that the pointer we pass to it is valid. It doesn't verify that.
- It expects that the argument it receives for the len parameter is the actual size of the array it operates on. A caller could pass a size that may cause out-of-bounds access.
- As it operates directly on a pointer, there is always a chance of out-of-bound access if pointer arithmetic is used in the function.

To eliminate these potential issues, in C++, instead of using a pointer to work on an array of data, we can use the class template std::span.

Span (view, not an owning container)

Standard: C++20.
What: Non-owning view over a contiguous block (T* + size). Lets you pass “a slice” without copying.

Ways to declare / define:

```cpp
#include <span>
#include <array>
#include <vector>

std::array<uint8_t, 256> buf{};
std::span<uint8_t> s1(buf);            // from std::array
std::span<uint8_t> s2(buf.data(), 128); // first half
std::vector<int> v{1,2,3,4};
std::span<const int> s3 = v;           // read-only span
```

Embedded tips:

- No allocation, no ownership → ideal for DMA buffers, register blocks, subranges.
- Safer than passing raw pointer + length; works great with constexpr helpers.

---

std::span is a lightweight, non-owning wrapper around a contiguous sequence of objects, where the first element is at position 0. It provides essential functionality such as the size() method, operator[] for element access, and the begin() and end() iterators, allowing it to integrate seamlessly with standard library algorithms.

Can be constructed from C-style arrays as well as containers like std::array and std::vector or etl::vector. This makes it a practical alternative to using separate pointer and size parameters, which is especially useful when interfacing C++ code with C libraries such as those used in HAL.

```cpp
#include <cstdio>
#include <span>

void print_ints(const std::span<int> arr) {
    for(int elem: arr) {
        printf("%d\r\n", elem);
    }
}

int main() {
```

```
    int arr[3] = {1, 2, 3};
    print_ints(arr);
    return 0;
}
```

In the above example, we can see that the function print_ints looks much simpler now. It accepts std::span of integers and it uses a range-based for loop to iterate over the elements. On the call site, we now just pass arr, an array of 3 integers. It is implicitly converted to std::span.

The class template std::span also has the size method, operator [], and begin and end iterators, meaning we can use it in standard library algorithms. We can also construct a subspan from span. It can be constructed from C-style arrays, but also from containers such as std::array and std::vector. It is a great solution to potential issues of interfaces that usually rely on pointer and size parameters.

List

Standard: C++98.
What: Doubly-linked list. Fast splices/insert/erase in the middle; no contiguous storage.

Ways to declare / define:

```
#include <list>

std::list<int> L1;
L1.push_back(1); L1.push_front(0);

std::list<int> L2{3,4,5};

std::list<int> L3(10, 7);  // 10 copies of 7

// splice
L1.splice(L1.end(), L2);   // move all nodes from L2 to L1 (O(1) per node link)
```

Embedded tips:

- Typically per-node dynamic allocation → memory fragmentation and cache-unfriendly.
- Avoid on small MCUs unless you have a node pool / custom allocator or use ETL's intrusive lists.

Deque

Standard: C++98.
What: Double-ended queue with efficient push_front/push_back. Internally segmented blocks; random access O(1) (slightly slower than vector).

Ways to declare / define:

```cpp
#include <deque>

std::deque<int> q = {1,2,3};
q.push_front(0);
q.push_back(4);
q.pop_front();
int mid = q[1];          // random access

// Use with algorithms
std::sort(q.begin(), q.end());
```

Embedded tips:

- Still uses dynamic allocation (manages blocks). If you need a ring buffer on MCUs, a fixed-size ring with std::array is usually better.

---
std::priority_queue is a container adapter that provides priority queue functionality. By default, it uses std::vector as the underlying container. However, you can substitute it with etl::vector from ETL, avoiding issues with dynamic memory allocation.

Associative containers (fast lookup with keys, sorted fashion)

Set

Standard: C++98.
What: Unique keys only, ordered. Ops: insert, find, lower_bound, erase are O(log n).

Ways to declare / define:

```cpp
#include <set>
```

```cpp
std::set<int> S = {3,1,4};
S.insert(2);
bool has3 = S.contains(3);        // C++20
auto it = S.find(4);
for (int x : S) { /* sorted iteration */ }
```

Embedded tips:

- Node allocations per element; heavier than arrays. Use when you must keep order and uniqueness without re-sorting.

Multiset

Standard: C++98.
What: Like set, but allows duplicates.

Ways to declare / define:

```cpp
#include <set>

std::multiset<int> MS = {1,1,2,3};
MS.insert(1);
auto range = MS.equal_range(1);   // all 1s
```

Embedded tips:

- Same memory/fragmentation caveats as set.

Map

Standard: C++98.
What: Ordered key→value (unique keys). Ops: operator[] inserts default value if key missing; at() bounds-checks; insert_or_assign (C++17).

Ways to declare / define:

```cpp
#include <map>
#include <string>

std::map<int, std::string> M;
```

```
M[10] = "ten";              // inserts if not present
M.insert({5, "five"});
auto it = M.find(10);
for (auto& [k,v] : M) { /* sorted by k */ }
```

Embedded tips:

- Each node allocates; consider std::array + std::sort + std::lower_bound for small, static maps to avoid heap.

---

std::map is an associative container that stores elements in key-value pairs. Each element has:

- A key (unique identifier)
- A value (data associated with the key)

The keys in a map are always unique and are stored in sorted order (by default, ascending, using < operator).

Key Characteristics of map:

1. Associative – Access elements by their key, not by position/index like arrays or vectors.
2. Ordered – Automatically sorts elements by key.
3. Unique keys – No two elements can have the same key.
4. Implemented as – Typically a self-balancing binary search tree (usually a Red-Black Tree).
5. Search, insert, delete operations – Have logarithmic complexity O(log n).

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    // Declare a map of int to string
    map<int, string> students;

    // Insert elements
    students[1] = "Alice";
    students[2] = "Bob";
    students[3] = "Charlie";

    // Another way to insert
    students.insert({4, "David"});
```

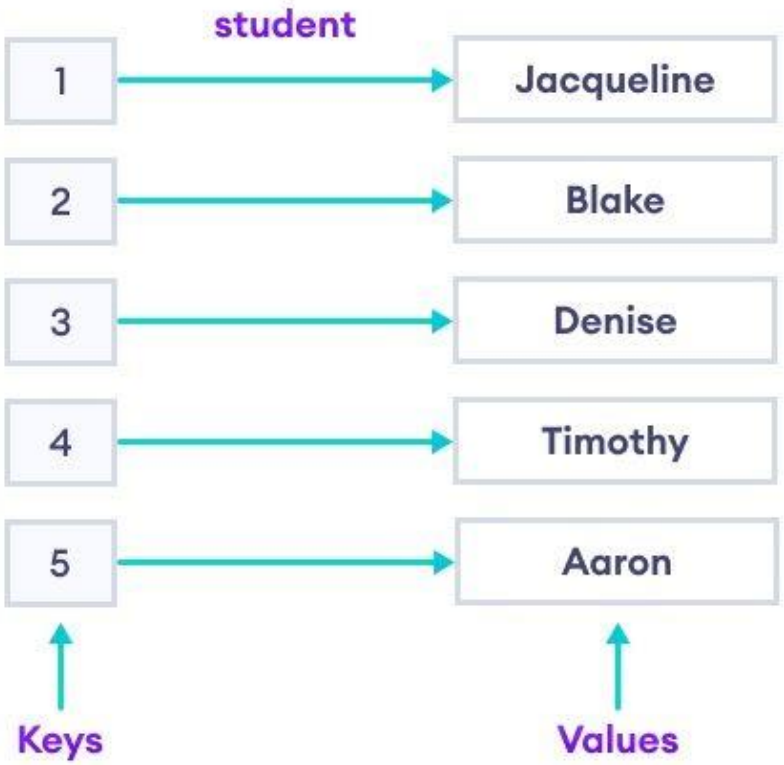
```
    // Access elements
    cout << "Student with roll 2: " << students[2] << endl;

    // Iterate through the map
    for (auto it : students) {
            cout << "Roll: " << it.first << ", Name: " << it.second << endl;
    }

    return 0;
}
```

Useful Functions:

- m.insert({key, value}); → insert element
- m[key]; → access or insert value by key
- m.find(key); → returns iterator to element or m.end() if not found
- m.erase(key); → remove element by key
- m.size(); → number of elements
- m.empty(); → check if empty



Multimap

Standard: C++98.
What: Ordered key→value with duplicate keys allowed.

Ways to declare / define:

```
#include <map>

std::multimap<int, const char*> MM = {{1,"a"},{1,"b"},{2,"c"}};
auto range = MM.equal_range(1);    // iterate both "a" and "b"
```

Embedded tips:

- Same as map; duplicates make sense for grouping, but memory-heavy on MCUs.

Unordered containers (hash-table-based)

Unordered_set

Standard: C++11.
What: Unique keys, hash-based. Ops: insert, find, contains (C++20), erase. Average O(1).

Ways to declare / define:

```cpp
#include <unordered_set>

std::unordered_set<int> US = {1,2,3};
US.insert(4);
bool has2 = US.contains(2);       // C++20
for (int x : US) { /* no order guarantee */ }
```

Embedded tips:

- Hash tables allocate buckets; can be RAM-heavy. If used, call reserve() with expected size to avoid rehashing.

Unordered_multiset

Standard: C++11.
What: Allows duplicates; hash-based.

Ways to declare / define:

```cpp
#include <unordered_set>

std::unordered_multiset<int> UMS = {1,1,2};
UMS.insert(1);
auto cnt = UMS.count(1);   // may be >1
```

Embedded tips:

- Same as above; duplicates further increase bucket pressure.

Unordered_map

Standard: C++11.
What: Hash map key→value (unique keys). It has average constant-time complexity for lookups, insertions, and deletions.

Ways to declare / define:

```cpp
#include <unordered_map>
#include <string>

std::unordered_map<std::string, int> UM;
UM.reserve(64);                    // important on embedded
UM["temp"] = 25;
UM.insert({"volt", 12});
if (UM.contains("temp")) { /* ... */ }

for (auto& [k,v] : UM) { /* no order */ }
```

Embedded tips:

- Excellent average-O(1) lookups, but: memory overhead, rehash cost, non-deterministic iteration order.
- Prefer fixed lookup tables or perfect hashing when determinism is critical.

Unordered_multimap

Standard: C++11.
What: Hash map with duplicate keys allowed.

Ways to declare / define:

```cpp
#include <unordered_map>

std::unordered_multimap<int, const char*> UMM = {{1,"a"},{1,"b"}};
auto [beg, end] = UMM.equal_range(1);   // iterate "a","b"
```

Embedded tips:

- Same caveats; duplicates increase memory. Consider separate vectors of values per key if you know an upper bound.

2. Algorithms: Process data

Algorithms from the standard library offer a consistent way to solve common problems across different containers, making the code more expressive and easier to maintain. They allow you to perform operations like searching, sorting, copying, and accumulating data using a uniform interface. Some of the most used algorithms are listed here:

Sorting

Sort

Introduced: C++98 (ranges version std::ranges::sort in C++20).

What it does: Sorts a range in ascending order by default. Typically O(N log N) (introsort). Requires Random Access Iterators (e.g., vector, deque, raw arrays).

```
#include <algorithm>
#include <vector>
#include <array>

std::vector<int> v = {3,1,4,1,5,9};
std::sort(v.begin(), v.end());                  // ascending

std::sort(v.begin(), v.end(), std::greater<int>()); // custom comparator

int a[] = {7,2,6,2};
std::sort(std::begin(a), std::end(a));          // C-array via helpers

std::array<int,5> ar{5,4,3,2,1};
std::sort(ar.begin(), ar.end());                // std::array
```

```
#include <algorithm>
#include <vector>
#include <functional>
#include <iostream>

int main() {
    std::vector<int> v = {4,2,5,1};

    // Using predefined functor to sort descending
```

```
    std::sort(v.begin(), v.end(), std::greater<int>());

    for (int x : v) std::cout << x << " "; // 5 4 2 1
}
```

Embedded notes: Fast and in-place; no allocations. Beware worst-case time for large N inside tight deadlines. Prefer std::array or bounded buffers to avoid reallocation.

---

std::sort Sorts a range of elements in ascending order by default, using the less-than operator for comparison. It can also accept a custom comparator to sort based on different criteria, such as descending order or a specific object property.

In the following example, we will generate elements randomly and sort them:

```
#include <cstdio>
#include <array>
#include <algorithm>
#include <random>

void print_container(const auto& container) {

    for(auto& elem: container) {
        printf("%d ", elem);
    }
    printf("\r\n");
}

int main() {

    std::array<int, 10> src{0};

    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 6);

    auto rand = [&](int x) -> int {
        return distrib(gen);
```

```
    };

    std::transform(src.begin(), src.end(), src.begin(), rand);
    print_container(src);

    std::sort(src.begin(), src.end());
    print_container(src);

    return 0;
}
```

In this example, we populate the src array using std::transform, which applies a rand lambda to every member of the src array. We used types from the random header to generate random numbers between 1 and 6. After we populate the array with random numbers, we sort it using std::sort. A possible output of this program is shown here:

6 6 1 1 6 5 4 4 1 1
1 1 1 1 4 4 5 6 6 6

We first see values in the array before sorting and then applying std::sort. We could have populated the initial array in a for loop, but we used the opportunity to demonstrate std:transform here.

Stable Sort

Introduced: C++98 (ranges std::ranges::stable_sort C++20).

What it does: Sorts while preserving the relative order of equal elements. Usually O(N log N); may use extra memory (not guaranteed in-place). Requires Random Access Iterators.

```
#include <algorithm>
#include <vector>
#include <string>

struct Item { int key; std::string tag; };
std::vector<Item> items = {{2,"a"},{1,"x"},{2,"b"}};

std::stable_sort(items.begin(), items.end(),
                 [](const Item& a, const Item& b){ return a.key < b.key; });
// Items with key==2 keep original "a" before "b"
```

Embedded notes: Use when tie-order matters (e.g., same timestamp). Can use extra RAM; check memory budget.

Searching

Binary Search

Introduced: C++98 (ranges std::ranges::binary_search C++20).

What it does: Checks existence of a value in a sorted range. Returns true/false. For position, use lower_bound/upper_bound. Requires Forward Iterators or better; typical O(log N).

```
#include <algorithm>
#include <vector>

std::vector<int> v = {1,2,4,4,5,9};
bool has4 = std::binary_search(v.begin(), v.end(), 4); // true

auto it = std::lower_bound(v.begin(), v.end(), 4);     // first <=4
auto it2 = std::upper_bound(v.begin(), v.end(), 4);    // first >4
```

Embedded notes: Pair with sorted std::array LUTs for deterministic and fast membership tests without hash tables.

Find

Introduced: C++98 (ranges std::ranges::find C++20).

What it does: Linear search for a value; returns iterator to first match or end. Works with Input Iterators and up.

```
#include <algorithm>
#include <array>

std::array<int,6> ar{3,1,4,1,5,9};

auto it = std::find(ar.begin(), ar.end(), 4);
```

```cpp
if (it != ar.end()) {
    // found at std::distance(ar.begin(), it)
}

auto it2 = std::find_if(ar.begin(), ar.end(), [](int x){ return x > 4; });
```

Embedded notes: O(N). Great for small, fixed buffers (e.g., scanning UART ring for a delimiter).
---

std::find: Searches for the first occurrence of a given value in a range and returns an iterator to it. If the value is not found, it returns the end iterator, signaling that the search failed.

Modifying

Copy

Introduced: C++98 (ranges std::ranges::copy C++20).

What it does: Copies a range to another (non-overlapping) range. Returns iterator to end of destination.

```cpp
#include <algorithm>
#include <array>

std::array<uint8_t,4> src{1,2,3,4};
std::array<uint8_t,4> dst{};
std::copy(src.begin(), src.end(), dst.begin());

// C arrays
int a[3] = {1,2,3}, b[3];
std::copy(std::begin(a), std::end(a), std::begin(b));
```

Embedded notes: Use for buffer moves in ISRs/tasks (non-overlap). For overlap, use std::copy_backward.
---

std::copy: Copies the elements of one range into another destination range.

Copy_if

Introduced: C++11 (ranges std::ranges::copy_if C++20).

What it does: Copies only elements satisfying a predicate. Returns end of written range.

```
#include <algorithm>
#include <vector>

std::vector<int> v = {1,2,3,4,5};
std::vector<int> out;
out.resize(v.size()); // ensure capacity
auto end_it = std::copy_if(v.begin(), v.end(), out.begin(),
                           [](int x){ return (x % 2) == 0; });
out.erase(end_it, out.end()); // shrink tail
```

Embedded notes: Useful for filtering samples into a preallocated buffer. Avoids dynamic growth if you size destination up-front.
---

std::copy_if: Copies only elements that satisfy a specified predicate, making it useful for filtering data as you copy.

Swap

Introduced: C++98 (move-aware & noexcept improvements in C++11).

What it does: Exchanges two objects.

```
#include <utility>

int a=1, b=2;
std::swap(a, b);

struct S { int x; };
S s1{10}, s2{20};
using std::swap; swap(s1, s2); // enables ADL for custom swaps
```

Embedded notes: Often compiles to a few moves. For large structs, consider custom swap to avoid copying (and mark noexcept).

Replace

Introduced: C++98 (ranges std::ranges::replace C++20).

What it does: Replaces all occurrences of a value in a range with a new value.

```
#include <algorithm>
#include <array>

std::array<int,6> ar{3,1,4,1,5,9};
std::replace(ar.begin(), ar.end(), 1, 42); // 1 -> 42

// With predicate
std::replace_if(ar.begin(), ar.end(), [](int x){ return x > 8; }, 8);
```

Embedded notes: Handy for sanitizing buffers (e.g., replace '\r' with '\n') in-place without extra RAM.

Counting

Count

Introduced: C++98 (ranges std::ranges::count C++20).

What it does: Counts how many elements equal a value. O(N).

```
#include <algorithm>
#include <array>

std::array<int,6> ar{3,1,4,1,5,9};
int ones = std::count(ar.begin(), ar.end(), 1); // -> 2
```

Embedded notes: Quick statistics over windows (e.g., how many samples exceed a threshold using count_if below).

Count_if

Introduced: C++98 (ranges std::ranges::count_if C++20).

What it does: Counts elements that satisfy a predicate. O(N).

```cpp
#include <algorithm>
#include <vector>

std::vector<int> v = {1,2,3,4,5};
int evens = std::count_if(v.begin(), v.end(), [](int x){ return (x&1)==0; });
```

Embedded notes: Use for lightweight signal quality checks (e.g., number of zero bytes in a packet).

Others

For_each

Introduced: C++98 (ranges std::ranges::for_each C++20).

What it does: Applies a function/functor to each element in a range; returns the function object by value.

```cpp
#include <algorithm>
#include <array>
#include <cstdint>

std::array<uint8_t,4> buf{1,2,3,4};
std::for_each(buf.begin(), buf.end(), [](uint8_t& x){ x += 1; }); // mutate

struct Sum { int s=0; void operator()(int x){ s+=x; } };
Sum f = std::for_each(buf.begin(), buf.end(), Sum{});
// f.s now has the sum
```

Embedded notes: Keeps loops concise; lambdas inline well. Avoid heavy work inside if running under tight deadlines.

---
std::for_each: Applies a specified function or lambda to each element in a range.

Min and Max

Introduced: C++98 (initializer-list overloads added C++11).

What they do: Return the smaller/larger of two values (or of an initializer list).

```cpp
#include <algorithm>
#include <initializer_list>

int a = std::min(3, 7);
int b = std::max({1,9,3,7});       // C++11 initializer_list
auto c = std::min(3.0, 3.0, std::less<double>()); // comparator form
```

Embedded notes: Great for clamping. For clamping a value, C++17 added std::clamp(val, lo, hi).

---

std::min and std::max: Return the smaller or larger of two values, respectively, using the less-than operator by default (or a provided comparison function). They're handy for quick comparisons where you just need the minimum or maximum of two values.

Min_element and max_element

Introduced: C++98 (ranges std::ranges::min_element / max_element C++20).

What they do: Return an iterator to the smallest/largest element in a range. O(N).

```cpp
#include <algorithm>
#include <array>

std::array<int,6> ar{3,1,4,1,5,9};
auto it_min = std::min_element(ar.begin(), ar.end()); // -> first 1
auto it_max = std::max_element(ar.begin(), ar.end()); // -> 9
```

Embedded notes: Useful for window stats without extra memory (e.g., find peak/valley in ADC window).

---

std::min_element and std::max_element: Return an iterator to the smallest or largest element in a range. These are useful when you need to find the position of an extreme value in a container (instead of comparing just two values).

Accumulate

Introduced: C++98.

What it does: Folds a range into a single value using a binary op (default: +). Lives in <numeric>.

```cpp
#include <numeric>
#include <array>

std::array<int,5> ar{1,2,3,4,5};
int sum = std::accumulate(ar.begin(), ar.end(), 0);      // 15

// With custom op (e.g., product)
int prod = std::accumulate(ar.begin(), ar.end(), 1,
                           [](int a, int b){ return a*b; });

// Accumulate into wider type to avoid overflow
long long sum64 = std::accumulate(ar.begin(), ar.end(), 0LL);
```

Embedded notes: Ideal for small windows (moving averages, checksum). Prefer wider accumulator types to avoid overflow; keep operations constant-time.

---

std::accumulate: Iterates over a range and combines the elements with an initial value using a binary operation (default is addition). This allows for summing values, computing products, or performing any custom aggregation you define.

3. Iterators: Access data

They act like pointers to elements of a container, bridge between containers and algorithms.

Types:
- Input
- Output
- Forward
- Bidirectional
- Random Access

For example: vector supports random access, but list only supports bidirectional iteration.

Iterators are abstractions that act like generalized pointers, providing a uniform way to traverse and access elements within a container. For example, standard library containers implement the begin() and end() methods, which return iterators marking the start and one-past-the-end of

their sequence. This consistent interface allows algorithms to work generically over different container types, enhancing code reusability and clarity.

```
#include <array>
#include <algorithm>
#include <cstdio>

int main() {
    std::array<int, 5> arr = {5, 3, 4, 1, 2};
    std::array<int, 5>::iterator start = arr.begin();
    auto finish = arr.end();
    std::sort(start, finish);
    for (auto it = arr.begin(); it != arr.end(); ++it) {
        printf("%d ", *it);
    }
    printf("\n");
    return 0;
}
```

How to use iterators with a standard library container:

• The iterator start is explicitly declared as std::array<int, 5>::iterator to illustrate the full type name, while the iterator finish is declared using auto for conciseness, allowing the compiler to deduce its type.
• The std::sort algorithm is applied using the iterators start and finish, obtained from arr.begin() and arr.end(), to sort the array in ascending order.
• The loop uses auto to declare the iterator it, which makes the code more concise. The loop traverses the sorted array, and printf is used to print each element.

Iterators are used to traverse containers. They not only promote generic programming but also make it easy to switch container types without changing the algorithmic logic.

Quick Capability Matrix (what operations each category guarantees)

Category	`*it` read	`*it = v` write	`++`	`--`	`it + n` / `it[n]`	Multi-pass
Input	✅	❌	✅	❌	❌	❌ single
Output	❌	✅	✅	❌	❌	❌ single
Forward	✅	✅ (if non-const)	✅	❌	❌	✅
Bidirectional	✅	✅ (if non-const)	✅	✅	❌	✅
Random Access	✅	✅ (if non-const)	✅	✅	✅	✅

(Const iterators allow read but not write.)

3.1 Input Iterator (single-pass, read-only)

Introduced: C++98 (category/concept), C++20 added the named ranges concept std::input_iterator.

What it does:

- Provides read-only sequential access (one-pass).
- Can move forward one step (++it), compare for equality (it1 == it2), and dereference (*it).
- Used when you just need to read a sequence once (e.g., reading from a stream).
- Used by algorithms like std::find, std::accumulate.

```
#include <algorithm>
#include <vector>
#include <sstream>   // for demo input stream (not typical on MCUs)
#include <iterator>

// Example 1: reading tokens from a stream via istream_iterator (classic input iterator)
void demo_stream_input_iterator() {
    std::istringstream iss("3 1 4 1 5");
    std::istream_iterator<int> first(iss);     // explicit type
    std::istream_iterator<int> last;           // default = end-of-stream
    int count_ones = std::count(first, last, 1); // single-pass, read-only
    (void)count_ones;
}

// Example 2: reading once from a buffer using raw pointers as input iterators
void demo_buffer_input() {
```

```
    const int buf[] = {10, 20, 30};
    const int* first = std::begin(buf);    // raw pointer is at least an input iterator
    const int* last  = std::end(buf);
    auto it = std::find(first, last, 20);  // works with input-iterator requirements
    (void)it;
}
```

Embedded C++ notes:

- Many bare-metal builds avoid iostreams; use raw pointers, std::array, or std::span instead.
- Input iterators are great for parsing a stream of bytes from a ring buffer: expose a pointer range (or a lightweight custom iterator) and feed it to algorithms like find, count, accumulate.
- Single-pass fits DMA/FIFO style reads.

3.2 Output Iterator (single-pass, write-only "sink")

Introduced: C++98 (output iterator and insert/stream iterators), C++20 adds std::output_iterator concept.

What it does:

- Lets you write elements sequentially to a destination (container, device, buffer).
- Supports: *it = value, ++it, it++. No meaningful *it read.
- Classic examples: std::back_inserter, std::ostream_iterator.

```
#include <algorithm>
#include <vector>
#include <iterator>
#include <iostream>

// Example 1: back_inserter writes into a vector (grows the container)
void demo_back_inserter() {
    std::vector<int> src {1,2,3};
    std::vector<int> dst;
    dst.reserve(src.size());
    std::copy(src.begin(), src.end(), std::back_inserter(dst)); // output iterator
}

// Example 2: ostream_iterator writes formatted output (not typical on MCUs)
```

```
void demo_ostream_iterator() {
    std::vector<int> v{10,20,30};
    std::ostream_iterator<int> out(std::cout, ",");
    std::copy(v.begin(), v.end(), out); // prints "10,20,30,"
}

// Example 3: raw buffer “output iterator” via pointers
void demo_pointer_output() {
    int outbuf[4]{};
    int* out = std::begin(outbuf);
    out = std::copy_n(std::begin(outbuf), 0, out); // no-op, but shows pattern
    *out++ = 42; *out++ = 7;                       // writing sequentially
}
```

Embedded C++ notes

- Prefer insert iterators (back_inserter) only if dynamic growth is allowed; otherwise write into fixed buffers with pointer output iterators.
- Great for format-free copy/filter pipelines: copy_if(src, dst_begin, pred) where dst_begin is a pointer into a statically allocated buffer.

3.3 Forward Iterator (multi-pass, forward only)

Introduced: C++98 (category), C++20 adds ranges concept std::forward_iterator.

What it does:

- Readable and writable, supports multiple passes (you can copy an iterator and iterate the same range again).
- Supports: *it, ++it, it++, ==/!=.
- Typical container: std::forward_list (singly linked). Many algorithms require at least forward iterators.

```
#include <forward_list>
#include <algorithm>

// Forward iterators from forward_list
void demo_forward_iterator() {
    std::forward_list<int> fl = {3,1,4,1,5};

    // Declaration styles
```

```
    std::forward_list<int>::iterator it = fl.begin();  // explicit type
    auto it2 = fl.begin();                             // with auto
    auto first = fl.begin(), last = fl.end();          // pair for algorithms

    // Multi-pass: you can traverse more than once
    int c1 = std::count(first, last, 1);
    int c2 = std::count(fl.begin(), fl.end(), 1);      // ok again
    (void)it; (void)it2; (void)c1; (void)c2;
}
```

Embedded C++ notes:

- forward_list uses per-node allocation → not ideal on tiny MCUs.
- Forward iterator requirements are helpful when you design custom intrusive singly-linked lists with static nodes (no heap) for RTOS queues—your custom iterators can meet "forward" semantics.

3.4 Bidirectional Iterator (multi-pass, forward and backward)

Introduced: C++98 (category), C++20 adds ranges concept std::bidirectional_iterator.

What it does:

- Everything a forward iterator does plus --it and it--.
- Containers: std::list, tree-based std::set/map/multiset/multimap.

```
#include <list>
#include <map>
#include <algorithm>
#include <string>

void demo_bidirectional_list() {
    std::list<int> L = {1,2,3};
    auto it = L.end();  // bidirectional: can move backward
    --it;               // now at 3
    --it;               // now at 2
}

void demo_bidirectional_map() {
    std::map<int, std::string> M{{1,"one"},{2,"two"},{3,"three"}};
```

```
    // explicit type
    std::map<int,std::string>::iterator it = M.begin();
    // auto style
    auto it2 = M.begin();

    // Walk forward and backward
    ++it;      // -> {2,"two"}
    --it;      // -> back to {1,"one"}
    (void)it2;
}
```

Embedded C++ notes:

- These containers allocate nodes; on MCUs prefer static pools or contiguous arrays when possible.
- Bidirectional capability is handy for LRU lists or history buffers—consider a custom double-linked list with nodes from a fixed pool to avoid fragmentation.

3.5 Random Access Iterator (multi-pass, jump anywhere in O(1))

Introduced: C++98 (category), C++20 adds ranges concept std::random_access_iterator.
(C++20 also recognizes contiguous_iterator—raw pointers, std::vector/array iterators qualify.)

What it does:

- All of bidirectional plus: it + n, it - n, it[n], <, <=, >, >=, and iterator difference (it2 - it1).
- Containers: std::vector, std::deque, std::array, and raw pointers.
- Enables fastest algorithms (e.g., std::sort needs random access).

```
#include <vector>
#include <array>
#include <algorithm>
#include <cstddef>

void demo_random_access_vector() {
    std::vector<int> v{3,1,4,1,5,9};

    // Declaration styles
    std::vector<int>::iterator it = v.begin(); // explicit
```

```
    auto it2 = v.begin();                  // auto
    int* p = v.data();                     // raw pointer

    it += 3;              // jump ahead
    int x = it[0];        // indexing via iterator
    std::ptrdiff_t dist = v.end() - v.begin(); // iterator difference

    std::sort(v.begin(), v.end());            // requires random access
    (void)it2; (void)p; (void)x; (void)dist;
}

void demo_random_access_array() {
    std::array<int,4> a{10,20,30,40};
    auto first = a.begin(), last = a.end();
    // pointer-style math works
    auto mid = first + 2;     // points to 30
    int second = first[1];    // 20
    (void)last; (void)mid; (void)second;
}
```

Embedded C++ notes:

- Raw pointers and std::array/std::span give you random access with zero overhead and deterministic memory—ideal for MCUs.
- Prefer contiguous storage when possible; it enables the broadest set of algorithms and best cache use.

4. Functors (Function Objects): Customize operations

Introduced:

- C++98: Functors and STL function objects (std::less, std::greater, etc.) introduced.
- C++11 onward: Lambdas became a shorter alternative, but functors remain useful, especially if state must persist.

A functor is simply a class or struct that overloads the function call operator operator(), so that its objects can be “called” like functions.

What it does:

- Objects that behave like functions.

- Can store state inside (unlike regular function pointers).
- Often used to customize STL algorithms (e.g., sorting order, predicates, transforms).
- Example: std::greater<int> is a functor used to sort in descending order.

Embedded C++ notes:

1. Why useful in embedded systems?
   - Functors allow compile-time, inline-able behavior (no vtable, no runtime dispatch if not virtual).
   - Unlike function pointers, they can hold configuration state (e.g., scale factor, threshold).
   - Often smaller and faster than polymorphism in resource-constrained MCUs.
   - Can be replaced with lambdas if available (C++11+).

2. Examples in embedded firmware:
   - Filter / scale sensor readings:

```
struct ScaleADC {
    int gain;
    int operator()(int sample) const { return sample * gain; }
};
```

   - Custom comparator for LUT search (sorted tables, calibration curves).
   - Event handlers where you need different behaviors but don't want full inheritance.
   - Replace repetitive loops with std::transform, std::for_each + functor.

3. Best practice in embedded:

Prefer functors/lambdas over virtual polymorphism for efficiency. They get inlined and generate deterministic, zero-overhead code if simple.

Code Examples:

Custom functor

```
#include <iostream>
#include <vector>
#include <algorithm>

// A custom functor that multiplies values
struct MultiplyBy {
```

```
    int factor;
    MultiplyBy(int f) : factor(f) {}

    // operator() makes this a functor
    int operator()(int x) const {
        return x * factor;
    }
};

int main() {
    std::vector<int> v = {1,2,3,4};

    // Using functor directly
    MultiplyBy times2(2);
    std::cout << times2(5) << "\n"; // prints 10

    // Using functor in an algorithm (transform)
    std::transform(v.begin(), v.end(), v.begin(), MultiplyBy(3));
    for (int x : v) std::cout << x << " "; // 3 6 9 12
}
```

STL predefined functors

```
#include <algorithm>
#include <vector>
#include <functional>
#include <iostream>

int main() {
    std::vector<int> v = {4,2,5,1};

    // Using predefined functor to sort descending
    std::sort(v.begin(), v.end(), std::greater<int>());

    for (int x : v) std::cout << x << " "; // 5 4 2 1
}
```

Lambdas as functors (modern)

```cpp
#include <algorithm>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v = {1,2,3,4};

    // Equivalent to writing a custom functor
    std::transform(v.begin(), v.end(), v.begin(),
                   [](int x){ return x * 10; });

    for (int x : v) std::cout << x << " "; // 10 20 30 40
}
```

Multiple declaration styles

```cpp
MultiplyBy m1(2);          // direct object
auto m2 = MultiplyBy(3);    // with auto
MultiplyBy m3 = {5};        // uniform init
int result = m1(10);        // call like a function
```

Embedded Template Library (ETL):

ETL provides a set of templated containers and algorithms that closely mimic the interfaces of standard library counterparts but are tailored for systems with limited resources.

One of the primary advantages of ETL is that its containers (such as etl::vector, etl::list, etl::string, and others) allow you to specify a fixed maximum size at compile time. Container implementations ensure that no dynamic memory allocation is performed at runtime as memory is reserved up front as atomic or static storage.

ETL also offers etl::array for platforms that do not support C++11, since std::array was introduced in C++11.

You can use etl::delegate instead of std::function to store a callable. However, etl::delegate is non-owning, so you must handle potential dangling references carefully.

ETL also provides utilities such as a messaging framework – a collection of messages, message routers, message buses, and finite state machines. It also offers CRC calculations, checksums, and hash functions.

ETL allows you to configure error handling. It can be configured to throw exceptions or send errors to the user-defined handler.

Advanced Concepts

1. Template

Introduced in: C++98 (with generic programming roots from C++ pre-98).

What they do: Allow writing generic code that works with many types, without code duplication. They enable type-safe compile-time polymorphism.

Templates allow writing generic functions and classes that can operate with any data type. Template classes and functions are defined using the template keyword, followed by the type parameter(s).

Code example:

```
template<typename T>
T max_val(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    int m1 = max_val(3, 7);       // int
    double m2 = max_val(3.5, 2.1); // double
}
```

Why Templates Are Great in C++:

1. Generic Programming:
    - Write one function/class and use it for any data type.
    - Example: std::vector<int> or std::vector<string> → same template class.
2. Type Safety:
    - Unlike void pointers in C, templates preserve type information.
    - Errors are caught at compile-time instead of runtime.

3. Code Reusability:
    - No need to duplicate functions for int, float, double.
    - One template works for all.
4. Performance (Zero Cost Abstraction):
    - Templates are resolved at compile time, so there's no runtime overhead.
    - As fast as writing type-specific functions.
5. Flexibility with STL:
    - Entire Standard Template Library (STL) (vector, map, set, queue) is built on templates.
    - Without templates, STL wouldn't exist in its current form.
6. Supports Metaprogramming:
    - Templates can be used for compile-time logic (Template Metaprogramming → TMP).
    - Example: computing factorial at compile time.

In Embedded C++:

✅ Best use: Zero-overhead polymorphism (compile-time), drivers, peripheral wrappers, signal processing.
⚠️ Can cause code bloat if instantiated too many times.

```
template<uint32_t BaseAddr>
struct Gpio {
    static void set() { *reinterpret_cast<volatile uint32_t*>(BaseAddr) |= 1; }
};

using LedPin = Gpio<0x48000014>; // port B ODR

int main() {
    LedPin::set(); // no runtime overhead
}
```

Embedded benefit: Each pin/peripheral is a type; optimized away by compiler.

Templates are used heavily in the C++ standard library. They allow us to implement the same functionality for different types, making our code reusable and generic, which is one of the strengths of C++.

In C++, templates serve as patterns or molds for functions and classes, allowing the creation of actual functions and classes. From this perspective, templates are not real functions or types themselves; rather, they act as guides for generating concrete functions and types.

```
#include <cstdio>
```

```
template<typename T>
T add(T a, T b) {
  return a + b;
}

int main() {
    int result_int = add(1, 4);
    float result_float = add(1.11f, 1.91f);

    printf("result_int = %d\r\n", result_int);
    printf("result_float = %.2f\r\n", result_float);

    return 0;
}
```

We have a template function, add, with the template type parameter T.

Template basics: Making a call to the template function

In this example, when the compiler sees a call to add a template function, it deduces the template argument and replaces the template parameter, in this case, type T, with type int in the first call and float in the second call to add.

After argument deduction, the template is instantiated; that is, the compiler creates two instances of the add function: one with integers as arguments and one with floats.

In assembly output, there are two instances of the add function: _Z3addIiET_S0_S0_, accepting integers, and _Z3addIfET_S0_S0_, accepting floats.
The compiler instantiated these two functions from the add template function, after it deduced template arguments on the call site of this function. This is the basic working principle of templates in C++.

Template basics: Template specialization

Template specialization allows us to provide the compiler with the implementation of a template function for a specific type:

```
template<>
point add<point>(point a, point b) {
```

```
    return point{a.x+b.x+1, a.y+b.y+1};
}
```

In this case, when the add function is called with arguments of type point, the compiler skips the generic template instantiation and uses this specialized version instead. This allows us to customize the behavior of the function specifically for point objects, adding an extra 1 to each coordinate when two point instances are added together. Let us take a look at the full main function now:

```
int main() {
    point a{1, 2};
    point b{2, 1};
    auto c = add(a, b);
    c.print();
    static_assert(std::is_same_v<decltype(c), point>);
    return 0;
}
```

We will get the following output: x = 4, y = 4
The compiler used function specialization for the point type. Template specialization makes templates a flexible tool, allowing us to provide compilers with custom implementations when needed.

2. Strong Types

Introduced gradually:

- enum class (scoped enums) → C++11
- explicit keyword → C++98
- Type-safe casts (static_cast, reinterpret_cast) → C++98

What it does: Prevents implicit, unsafe conversions and avoids bugs caused by weak typing.

Code example:

```
enum class Color { Red, Green, Blue };

Color c = Color::Red;
// int x = c; // ❌ not allowed
int x = static_cast<int>(c); // ✅ explicit
```

In Embedded C++:

✅ Best use: Prevent mixing units (milliseconds vs ticks), or pins vs channels.
⚠️ Too many strong typedefs may add verbosity.

```
enum class AdcChannel : uint8_t { Temp = 16, Vref = 17 };

void start_conversion(AdcChannel ch);

start_conversion(AdcChannel::Temp); // ✅ type safe
```

Embedded benefit: Stops “wrong channel number” bugs at compile time.

3. Lambdas

Introduced in: C++11
Enhanced in C++14 (generalized captures)
C++17 (constexpr lambdas)
C++20 (template lambdas)

What they do: Define anonymous functions (a function without a name) inline (can define directly inside your code), often used for callbacks, algorithms, or embedded ISR handlers.

[capture_list](parameters) -> return_type { body };

Code example:

```
auto square = [](int x) { return x * x; };

int main() {
    int result = square(5);  // 25
}
```

In Embedded C++:

✅ Best use: Small callbacks for ISRs, RTOS tasks, button handling.
⚠️ Avoid capturing heap objects; keep them constexpr or small.

```
auto is_button_pressed = []() { return (GPIOA->IDR & (1<<0)); };

if (is_button_pressed()) {
    // do something
}
```

Embedded benefit: Compact, inline callbacks without needing full function objects.

A generic lambda is a lambda whose parameter types are deduced from the call, i.e. the lambda works like a function template. Introduced in C++14 (using auto in parameter list) and extended in C++20 (allowing an explicit template parameter list []<...>(...)), generic lambdas let you write concise, type-polymorphic callables while still capturing local state.

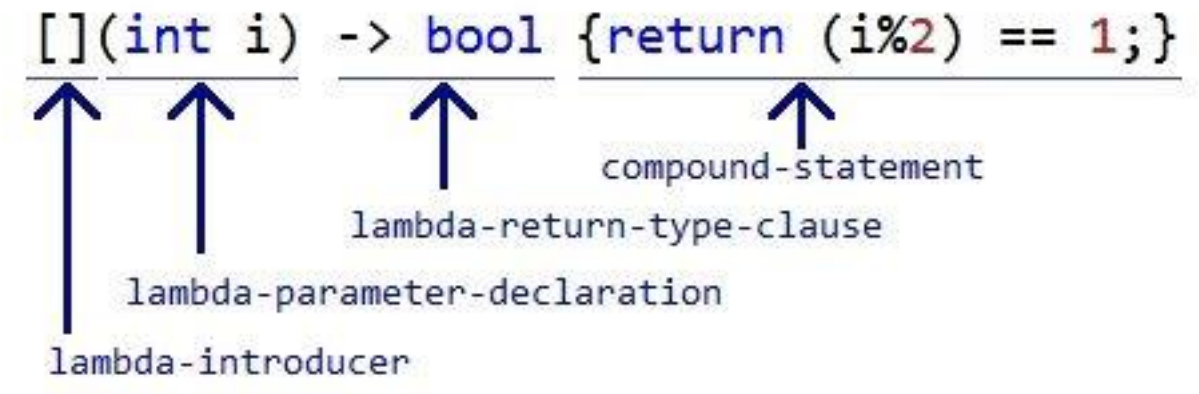
```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
 vector<int> v = {1, 2, 3, 4, 5};

 // Use lambda inside for_each
 for_each(v.begin(), v.end(), [](int n) {
 cout << n * n << " ";
 });
}
```

Lambdas are objects with an overloaded operator() (functors).

- The compiler generates a unique type for each lambda.
- They are useful in algorithms, callbacks, and small utilities.
- With C++14/20, lambdas became generic (with auto or template parameters).



	C++11/C++14	C++17	C++20
[this]	&(*this)	&(*this)	&(*this)
[*this]	X	*this	*this
[this, *this]	X	X	X
[&]	Implicit &(*this)	Implicit &(*this)	Implicit &(*this)
[&, this]	&(*this)	&(*this)	&(*this)
[&, *this]	X	*this	*this
[=]	Implicit &(*this)	Implicit &(*this)	Deprecated Implicit &(*this)
[=, this]	X	X	&(*this)
[=, *this]	X	*this	*this

X - Invalid (Error or Warning) C++17 Change C++20 Change

nextptr

Lambda expressions, or lambdas, were introduced in C++11.

Lambda expressions in C++ allow us to write short blocks of code that encapsulate functionality and capture the surrounding state into a callable object. We can use operator() on a callable object to execute the functionality implemented in it.

Common uses of lambdas include passing a function object (also called a functor – an object of a class that overrides operator()) to standard library algorithms, or any code expecting a function object, encapsulating small blocks of code that are often used only in a single function, and variable initialization.

In embedded development, lambdas are especially useful for defining actions in response to timer or external interrupts, scheduling tasks, and similar event-driven mechanisms.

They are used to create an instance of an unnamed closure type in C++. A closure stores an unnamed function and can capture variables from its scope by value or reference.

We can call operator () on a lambda instance, with arguments specified in the lambda definition, effectively calling the underlying unnamed function.

Example:

```
int main(){
    std::array<int, 4> arr{5, 3, 4, 1};

    const auto print_arr = [&arr](const char* message) {
        printf("%s\r\n", message);
        for(auto elem : arr) {
            printf("%d, ", elem);
        }
        printf("\r\n");
    };

    print_arr("Unsorted array:");

    std::sort(arr.begin(), arr.end(), [](int a, int b) {return a < b;});
    print_arr("Sorted in ascending order:");

    std::sort(arr.begin(), arr.end(), [](int a, int b) {return a > b;});
    print_arr("Sorted in descending order:");

    return 0;
}
```

Output:
Unsorted array:
5, 3, 4, 1,
Sorted in ascending order:
1, 3, 4, 5,

Sorted in descending order:
5, 4, 3, 1,

Lambda print_arr used to print an array arr:

• The [&arr] syntax captures the variable arr by reference from the surrounding scope. This means the lambda can access and use arr directly within its body.

• We can capture variables by value, or by reference if we prefix the name of a variable with & as we did for the print_arr lambda.

• Capturing by reference [&arr] allows the lambda to see any changes made to arr outside the lambda after its definition. If we captured by value, the lambda would have its own copy of arr.

• By defining print_arr as a lambda within main, we encapsulate the functionality of printing the array without needing to create a separate function. This keeps related code together and enhances readability.

Lambdas as predicate functions

- The std::sort algorithm rearranges the elements of arr based on the comparator provided.
- The lambda [](int a, int b) { return a < b; } acts as a comparator function for std::sort. It takes two integers and returns true if the first is less than the second, which results in an ascending sort.
- The lambda [](int a, int b) { return a > b; } returns true if the first integer is greater than the second, resulting in a descending sort.

We can also specify the lambda return type explicitly as in the following example:

```
auto greater_than = [](int a, int b) -> bool {
return a > b;
};
```

Here, we explicitly defined the return type. This is optional and can be used when we want to be explicit about the type that a lambda returns.

Also, note that the capture clause of this lambda is empty square brackets []. This indicates that the lambda is not capturing any variables from the surrounding scope.

When the lambda is capturing a variable by reference, it is important to note that this introduces lifetime dependency – meaning that the object that reference is bound to must exist when we call the lambda – else, we will use a so-called dangling reference, which is undefined behavior.

This is especially a concern with asynchronous operations – that is, when a lambda is passed to a function and called later.

Storing lambdas using std::function

std::function is a class template that allows us to store, copy, and invoke callable objects such as function pointers and lambdas.

Example:

```
#include <cstdio>
#include <cstdint>
#include <functional>

int main() {
    std::function<void()> fun;
    fun = []() {
        printf("This is a lambda!\r\n");
    };
    fun();

    std::uint32_t reg = 0x12345678;
    fun = [reg]() {
        printf("Reg content 0x%8X\r\n", reg);
    };
    reg = 0;
    fun();

    return 0;
}
```

• In the main function, we first create an object fun of type std::function<void()>. This specifies that fun can store any callable object that returns void and takes no arguments. This includes function pointers, lambdas, or any object with an operator() that matches the signature.

• We then assign a lambda to fun and invoke it, which prints the message “This is a lambda!” to the console.

• Next, we assigned another lambda to the fun object. This time the lambda captures the uint32_t reg by value from the surrounding scope and prints it. Capturing by value means the lambda makes its own copy of reg at the moment the lambda is defined.

• We change the value of reg to 0 before invoking the callable object stored in fun to show it is being captured by value. Calling fun prints Reg content 0x12345678.

4. Smart Pointers

Introduced in:

- std::auto_ptr → C++98 (deprecated in C++11)
- std::unique_ptr, std::shared_ptr, std::weak_ptr → C++11

What they do: A smart pointer is an object in C++ that automatically manages the lifetime of dynamically allocated memory. Smart pointers help avoid memory leaks and dangling pointers by automatically deallocating memory when it is no longer needed.
- unique_ptr: sole ownership.
- shared_ptr: reference-counted ownership.
- weak_ptr: non-owning reference (avoids cycles).

Code example:

```
#include <memory>
#include <iostream>

int main() {
    std::unique_ptr<int> p1 = std::make_unique<int>(42);
    std::cout << *p1 << "\n"; // prints 42
}
----
```

When to use what (quick map)

Situation	Recommended
You need exclusive ownership, lightweight, deterministic destruction	`unique_ptr`
Multiple owners must share lifetime; you can afford overhead	`shared_ptr` (use `make_shared`)
You need to observe a `shared_ptr` without extending its life / avoid cycles	`weak_ptr`
Legacy code mentions `auto_ptr`	Replace with `unique_ptr`

In Embedded C++:

- Prefer static/stack allocation where possible (no smart pointer needed).
- If you must use dynamic allocation:
  - Favor unique_ptr; pre-allocate outside real-time paths.
  - Consider custom deleters that return objects to fixed pools (no free), keeping determinism.
  - Never allocate/free in ISRs; pass references or raw, non-owning pointers into interrupt code.
- On RTOS/embedded Linux layers, shared_ptr can be fine, but measure:
  - Refcount updates may be atomic (heavier).
  - Destruction may happen in unexpected contexts—avoid holding locks or scarce HW resources inside destructors.

✅ Best use (cautious): In embedded Linux, RTOS, or higher-end MCUs (ARM Cortex-A).
⚠️ On bare-metal Cortex-M: avoid heap in ISRs; unique_ptr is OK if you never use new in ISRs. Prefer static allocation.

```
#include <memory>
struct UartDriver { void init(){} };

int main() {
    auto uart = std::make_unique<UartDriver>();
    uart->init();
}
```

Embedded benefit: Automatic cleanup (RAII) for dynamically allocated drivers — but only if dynamic allocation policy is safe in your RTOS.

In C++, if you use raw pointers (new / delete), you're responsible for freeing memory. Forgetting delete causes memory leaks.
Smart pointers automate this:

- std::unique_ptr<T> → owns a resource exclusively, auto-deletes when it goes out of scope.
- std::shared_ptr<T> → reference-counted ownership, deletes only when last reference is gone.
- std::weak_ptr<T> → non-owning observer to a shared_ptr.

```
#include <iostream>
#include <memory>
using namespace std;

struct Node {
 int data;
 unique_ptr<Node> next; // smart pointer instead of raw Node*
 Node(int v) : data(v) {}
};

int main() {
 auto head = make_unique<Node>(10);
 head->next = make_unique<Node>(20);
 head->next->next = make_unique<Node>(30);

 cout << head->data << " -> "
 << head->next->data << " -> "
 << head->next->next->data << endl;

 // No delete needed, memory auto-freed
}
```

Above program ensures no risk of memory leak, no manual delete.

a) unique_ptr: sole ownership (no copies)

Introduced: C++11.

Helper: std::make_unique in C++14.

What it does: Owns an object exclusively. Non-copyable; movable. Deletes the object when it goes out of scope (RAII). Lightweight (no refcount). Can hold arrays (unique_ptr<T[]>) and custom deleters (type is part of the pointer type).

Common declarations & usage:

```cpp
#include <memory>
#include <vector>
#include <cstdio>

// Basic ownership
std::unique_ptr<int> a = std::make_unique<int>(42);  // C++14+
auto b = std::make_unique<std::pair<int,int>>(1,2);  // CTAD friendly via 'auto'

// Moving ownership
auto c = std::move(a);       // a becomes null, c owns 42
if (!a) { /* a is empty */ }

// Custom deleter (e.g., fclose a FILE*)
struct FileCloser { void operator()(std::FILE* f) const noexcept { if (f) std::fclose(f); } };
using FilePtr = std::unique_ptr<std::FILE, FileCloser>;
FilePtr logFile(std::fopen("log.txt","w")); // auto-closed on scope exit

// Array form
auto buf = std::make_unique<uint8_t[]>(256);
buf[0] = 0xAA;

// Polymorphism
struct Base { virtual ~Base() = default; };
struct Der : Base {};
std::unique_ptr<Base> p = std::make_unique<Der>();   // OK: deletes via Base::~Base()

// Containers of unique_ptr (owning collections)
std::vector<std::unique_ptr<Base>> v;
v.push_back(std::make_unique<Der>());
```

Embedded C++ notes:

- ✅ Best smart pointer for MCUs if you ever use dynamic allocation: no refcount, minimal overhead.
- Prefer make_unique for exception safety and smaller codegen.
- If the heap is disallowed, you can still use unique_ptr with a custom deleter that returns objects to a static pool (no delete!).
- Never allocate/free in ISRs. Create before ISR, pass raw non-owning pointers or references into the ISR.

b) shared_ptr: reference-counted ownership

Introduced: C++11.
Helper: std::make_shared (C++11) allocates object + control block together (often smaller/faster).

What it does: Multiple shared_ptrs share ownership of the same object via a control block (stores refcount and deleter). Object is destroyed when the last shared_ptr goes away. Can create cycles (which leak) unless broken with weak_ptr.

Common declarations & usage:

```
#include <memory>
#include <string>

// Basic usage
auto sp1 = std::make_shared<std::string>("hello"); // refcount = 1
auto sp2 = sp1;                                    // refcount = 2
sp2.reset();                                       // refcount = 1

// Custom deleter (rarely needed with make_shared)
auto raw = new int(7);
std::shared_ptr<int> sp3(raw, [](int* p){ /* custom cleanup */ delete p; });

// Beware cycles:
struct Node {
    std::shared_ptr<Node> next;   // ✕ can create cycles and leak
    // fix: use weak_ptr<Node> next;
};
```

Polymorphism & factories:

```cpp
struct Base { virtual ~Base() = default; };
struct Der : Base {};
std::shared_ptr<Base> make() { return std::make_shared<Der>(); }
```

Embedded C++ notes:

- ⚠ Overhead: refcount updates are typically atomic → code size/time cost, especially on SMP/RTOS.
- ⚠ Non-deterministic deallocation time (happens when last owner disappears)—can surprise real-time paths.
- Prefer unique_ptr + explicit ownership transfer. Use shared_ptr sparingly (e.g., app layer on embedded Linux, or where sharing is unavoidable).
- Always consider whether a plain reference / std::span / observer is enough.

c) weak_ptr: non-owning observer (breaks cycles)

Introduced: C++11
Purpose: Prevent cycles and allow observers that don't extend lifetime.

What it does: Points to a shared_ptr-managed object without owning it. Doesn't change refcount. You must call .lock() to get a shared_ptr before use; if the object is already destroyed, lock() returns an empty shared_ptr.

Common declarations & usage:

```cpp
#include <memory>
#include <iostream>

struct Node {
    std::shared_ptr<Node> child;
    std::weak_ptr<Node>   parent; // non-owning to avoid cycle
};

auto root = std::make_shared<Node>();
auto leaf = std::make_shared<Node>();
root->child = leaf;
leaf->parent = root;  // no cycle leak

// Using weak_ptr safely
```

```cpp
if (auto p = leaf->parent.lock()) {       // try to promote to shared_ptr
    // p is a valid shared_ptr while in scope
} else {
    // parent no longer exists
}
```

Embedded C++ notes:

- The cost is essentially that of shared_ptr's control block, but weak_ptr doesn't change strong refcount.
- Useful where you must observe (GUI, pub-sub, async callbacks) without keeping objects alive.
- For tiny MCUs, consider non-owning raw pointers with clear lifetimes (simpler, but requires discipline).

5. Virtual Tables (vtables)

Introduced in: C++98 (fundamental OOP feature since the beginning).

What they do: Enable runtime polymorphism by keeping a hidden table of function pointers for each polymorphic class.
When you call a virtual method on a base pointer, the vtable is consulted to find the right override.

Code example:

```cpp
#include <iostream>
class Base {
public:
    virtual void speak() { std::cout << "Base\n"; }
};
class Derived : public Base {
public:
    void speak() override { std::cout << "Derived\n"; }
};

int main() {
    Base* obj = new Derived();
    obj->speak(); // Calls Derived::speak() via vtable
    delete obj;
```

```
}
```

In Embedded C++:

✅ Best use: When you need runtime polymorphism (e.g., multiple sensors on same bus).
⚠️ Adds flash & RAM overhead (vtable + vptr per object). Can impact ISRs.
👉 Alternative: use templates (CRTP) for static polymorphism if performance critical.

```
class ISensor {
public:
    virtual int read() = 0;
};

class TempSensor : public ISensor {
public:
    int read() override { return adc_read(); }
};

void sample(ISensor& s) {
    int val = s.read(); // uses vtable
}
```

Embedded benefit: Easier driver swapping at runtime, but heavier than templates.

6. RAII: Resource Acquisition Is Initialization (principle)

It's a C++ programming idiom where resource management (like memory, file handles, locks, sockets) is tied to the lifetime of an object.

- The resource is acquired in the constructor.
- The resource is released in the destructor.

This ensures automatic cleanup when the object goes out of scope (even in case of exceptions).

Instead of manually calling acquire() and release(), you wrap the resource in a class.

- Constructor = acquires resource.
- Destructor = releases resource.

By automatically managing resource allocation and deallocation, RAII greatly reduces the risk of leaks and other resource misuse issues, which is especially important in resource-limited embedded environments.

```cpp
#include <iostream>

class RAIIArray {
    int* data;
public:
    RAIIArray(size_t size) {
    data = new int[size]; // Resource acquired
    std::cout << "Allocated " << size << " ints\n";
    }
    ~RAIIArray() {
    delete[] data; // Resource released
    std::cout << "Memory freed\n";
    }
};

int main() {
    {
    RAIIArray arr(10); // allocates
    } // arr goes out of scope → destructor called → memory freed automatically
}
```

No memory leak, even if exceptions occur.

Everyday RAII in C++:

- std::string (manages dynamic char arrays)
- std::vector (manages dynamic arrays)
- std::shared_ptr / std::unique_ptr (manage heap memory)
- std::thread (joins/detaches on destruction)
- std::fstream (closes file in destructor)
- std::lock_guard / std::unique_lock (release lock automatically)

Summary:

- RAII ties resource lifetime to object lifetime.
- Resource acquired in constructor, released in destructor.
- Ensures no leaks and exception safety.
- Foundation of modern C++ (smart pointers, containers, locks, etc).

RAII is a simple yet powerful C++ technique used to manage resources through an object's lifetime. Resources can represent different things. Resources are acquired when an object's lifetime begins, and they are released when the object's lifetime ends.

The technique is used to manage resources such as dynamically allocated memory. To ensure that the memory is released and avoid memory leaks, the recommendation is to **use dynamic allocation only internally in classes**. When an object is instantiated, the constructor will allocate memory, and when the object goes out of scope, the destructor will release the allocated memory.

The RAII technique can be applied to other resources beyond the dynamically allocated memory, such as files.

7. Finite State Machines

A Finite State Machine (FSM) is an abstract computational module used to represent a system that can be in exactly one of a finite number of states at any given time. An FSM can transition from one state to another on a given input, and it can perform an action during the transition.

SOLID Design Principles

SOLID principles are a set of guidelines or best practices for object-oriented programming that help to create software that is more maintainable, flexible, and scalable. The term "SOLID" is an acronym for five different principles, each of which focuses on a different aspect of software development:

1. Single Responsibility Principle (SRP):

A class should have only one responsibility, giving it a single reason to change.

In C++, SRP can be implemented by dividing the responsibilities of a class into smaller, more focused classes. Each class should be responsible for only one task and should have no other reason to change. This makes the code easier to understand, modify and test.

To implement SRP, we can create separate classes for each of these responsibilities.

For example, we can create a CustomerDetails class to handle the customer details, a BillingCalculator class to calculate the final bill, and an InvoiceGenerator class to generate the invoice.

```
class CustomerDetails {
```

```
private:
  string name;
  int id;
public:
  void setName(string name);
  string getName();
  void setId(int id);
  int getId();
};

class BillingCalculator {
private:
  vector<Item> items;
public:
  void addItem(Item item);
  void removeItem(Item item);
  float calculateTotalAmount();
};

class InvoiceGenerator {
public:
  string generateInvoice(CustomerDetails customerDetails, BillingCalculator billingCalculator);
};
```

By dividing the responsibilities of the Customer class into smaller, more focused classes, each class has only one responsibility, which makes the code more maintainable and scalable. If any changes are required in any of these classes, only that class needs to be modified, rather than the entire Customer class. This leads to cleaner and more maintainable code, which is easier to understand, modify and test.

2. Open/Closed Principle (OCP):

A class should be open for extension but closed for modification. A new functionality is added by extending the class through dynamic or static polymorphism, rather than modifying it.

In C++, OCP can be implemented by creating classes that can be extended by adding new functionality without modifying the existing code. This is achieved by using inheritance, polymorphism, and interfaces. By following this principle, we can make the code more modular and less prone to bugs.

Let’s consider an example of a Shape class hierarchy that includes different types of shapes, such as Rectangle, Circle, and Triangle. Each shape has a different area calculation method.

```
class Shape {
public:
  virtual double area() = 0;
};

class Rectangle : public Shape {
private:
  double width;
  double height;
public:
  Rectangle(double width, double height);
  double area() override;
};

class Circle : public Shape {
private:
  double radius;
public:
  Circle(double radius);
  double area() override;
};

class Triangle : public Shape {
private:
  double base;
  double height;
public:
  Triangle(double base, double height);
  double area() override;
};
```

Now, let’s suppose that we want to add a new shape, such as a Square, to our hierarchy. One way to do this would be to modify the Shape class hierarchy by adding a new Square class. However, this would violate the OCP, as we would be modifying the existing code.

Instead, we can extend the Shape class hierarchy by creating a new class that inherits from the Shape class and implements the area() method. In this way, we are adding new functionality without modifying the existing code.

```
class Square : public Shape {
private:
  double side;
public:
  Square(double side);
  double area() override;
};
```

Now, we can create objects of the Square class and calculate their areas using the area() method without modifying the existing code.

```
Square square(5.0);
double squareArea = square.area(); // returns 25.0
```

By following the Open-Closed Principle, we can create code that is more maintainable, flexible, and easier to extend. This allows us to add new functionality to our code without modifying the existing code, which reduces the risk of introducing bugs and makes the code more modular.

3. Liskov Substitution Principle (LSP):

Derived classes should be usable in place of their parent classes without breaking the software's behavior.

In C++, LSP can be implemented by ensuring that subclasses adhere to the same interface and behavior as their parent class. This allows us to use a subclass object wherever we would normally use a parent class object.

Let's consider an example of a Bird class hierarchy that includes different types of birds, such as Eagle, Penguin, and Ostrich. Each bird has a different ability to fly.

```
class Bird {
public:
  virtual void fly() = 0;
};

class Eagle : public Bird {
```

```
public:
  void fly() override;
};

class Penguin : public Bird {
public:
  void fly() override;
};

class Ostrich : public Bird {
public:
  void fly() override;
};
```

Now, let's suppose that we have a function that takes a Bird object and makes it fly.

```
void makeBirdFly(Bird& bird) {
  bird.fly();
}
```

According to the LSP, we should be able to pass any Bird subclass object to this function without affecting the correctness of the program. For example, we should be able to pass an Eagle object, a Penguin object, or an Ostrich object to this function.

```
Eagle eagle;
Penguin penguin;
Ostrich ostrich;

makeBirdFly(eagle); // okay, eagle can fly
makeBirdFly(penguin); // error, penguin can't fly
makeBirdFly(ostrich); // error, ostrich can't fly
```

As we can see, passing a Penguin or an Ostrich object to this function would result in an error because they cannot fly. Therefore, these classes do not adhere to the same behavior as their parent class, Bird, and violate the LSP.

By following the Liskov Substitution Principle, we can create code that is more modular and flexible. This allows us to use subclasses interchangeably with their parent class, which reduces the complexity of our code and makes it easier to maintain.

4. Interface Segregation Principle (ISP):

Interface classes should remain small and concise so that derived classes implement only methods they need.

In C++, ISP can be implemented by breaking down larger interfaces into smaller, more specific interfaces. This allows clients to only depend on the specific methods they need, rather than being forced to implement unnecessary methods.

Let's consider an example of a Printer class that can print documents in different formats, such as PDF, HTML, and plain text. We could define a single interface for this class that includes all possible methods, such as printPDF(), printHTML(), and printPlainText().

```
class Printer {
public:
  virtual void printPDF() = 0;
  virtual void printHTML() = 0;
  virtual void printPlainText() = 0;
};
```

However, this interface may not be suitable for all clients. For example, a client that only needs to print plain text documents may not want to implement the printPDF() and printHTML() methods.

To adhere to the ISP, we could break down the Printer interface into smaller interfaces that are specific to each document format.

```
class Printable {
public:
  virtual void print() = 0;
};

class PDFPrintable : public Printable {
public:
  void print() override;
};

class HTMLPrintable : public Printable {
public:
```

```
  void print() override;
};

class PlainTextPrintable : public Printable {
public:
  void print() override;
};
```

Now, clients can depend on the specific interfaces they need. For example, a client that only needs to print plain text documents can depend on the PlainTextPrintable interface.

```
class PlainTextPrinter {
public:
  void print(PlainTextPrintable& document) {
    document.print();
  }
};
```

By adhering to the Interface Segregation Principle, we can create more flexible and reusable code that is easier to maintain. Clients can depend on smaller interfaces that are specific to their needs, rather than being forced to implement unnecessary methods.

5. Dependency Inversion Principle (DIP):

High-level modules (e.g., an accelerometer) should not depend on low-level modules (e.g., I2C). Both should rely on abstractions (interfaces) rather than concrete implementations.

In C++, DIP can be implemented by using interfaces or abstract classes to decouple high-level modules from low-level modules. This allows for more flexibility in the design, as different implementations of the same interface can be easily swapped out without affecting the overall structure of the program.

Let's consider an example of a Database class that stores user information. We could define a concrete implementation of this class that depends on the MySQLConnector class.

```
class MySQLConnector {
public:
  void connect();
  void query(const std::string& query);
};
```

```
class Database {
public:
  void addUser(const std::string& name, const std::string& email) {
    MySQLConnector connector;
    connector.connect();
    std::string query = "INSERT INTO users (name, email) VALUES ('" + name + "', '" + email + "')";
    connector.query(query);
  }
};
```

This implementation violates the Dependency Inversion Principle, as the Database class depends on the low-level MySQLConnector class. If we were to switch to a different database system, we would need to modify the Database class to use a different connector.

To adhere to the DIP, we could define an abstract interface for the database connector, and have the Database class depend on this interface instead of a concrete implementation.

```
class IDatabaseConnector {
public:
  virtual void connect() = 0;
  virtual void query(const std::string& query) = 0;
};

class MySQLConnector : public IDatabaseConnector {
public:
  void connect() override;
  void query(const std::string& query) override;
};

class Database {
public:
  Database(IDatabaseConnector& connector) : connector_(connector) {}

  void addUser(const std::string& name, const std::string& email) {
    connector_.connect();
    std::string query = "INSERT INTO users (name, email) VALUES ('" + name + "', '" + email + "')";
    connector_.query(query);
```

```
  }

private:
  IDatabaseConnector& connector_;
};
```

Now, the Database class depends on the abstract IDatabaseConnector interface instead of a concrete implementation. We can create different implementations of the IDatabaseConnector interface for different database systems, and easily swap them out without modifying the Database class.

```
class PostgreSQLConnector : public IDatabaseConnector {
public:
  void connect() override;
  void query(const std::string& query) override;
};

int main() {
  PostgreSQLConnector postgresConnector;
  Database db(postgresConnector);

  db.addUser("John Doe", "john.doe@example.com");
}
```

By adhering to the Dependency Inversion Principle, we can create more flexible and maintainable code that is easier to extend and modify. High-level modules depend on abstractions, not low-level details, which allows for easier substitution of implementations.

Design Patterns

Design patterns are typical solutions to commonly occurring problems in software design. They are like pre-made blueprints that you can customize to solve a recurring design problem in your code.

Patterns are often confused with algorithms, because both concepts describe typical solutions to some known problems. While an algorithm always defines a clear set of actions that can achieve some goal, a pattern is a more high-level description of a solution. The code of the same pattern applied to two different programs may be different.

Most patterns are described very formally so people can reproduce them in many contexts. Here are the sections that are usually present in a pattern description:

- Intent of the pattern briefly describes both the problem and the solution.
- Motivation further explains the problem and the solution the pattern makes possible.
- Structure of classes shows each part of the pattern and how they are related.
- Code example in one of the popular programming languages makes it easier to grasp the idea behind the pattern.

Quick embedded guidance (cheat-sheet)

Pattern	Use it for	Embedded tips
Singleton	Truly unique resource	Prefer function-local static; avoid hidden deps; DI is often better
Adapter	Portability over HALs	Interface at app layer + adapter per MCU; mock in tests
Command	Defer/queue work	ISR → queue → worker; use static pool/ring buffer
Observer	Event fan-out	Pre-register callbacks; keep ISR tiny; consider lock-free queues
State	Clear mode logic	Table/enum for zero overhead; measure transition time

Creational Patterns

Provide object creation mechanisms that increase flexibility and reuse of existing code.

Singleton

What it does: Ensures there's exactly one instance of a type and provides a global access point to it (e.g., a board-wide logger or a unique hardware peripheral). Useful when the resource is truly unique.

Caveats: hidden dependencies, hard to test, order-of-init pitfalls. In embedded, prefer explicit wiring unless the resource is inherently unique.

```
// "Meyers Singleton": lazy, thread-safe since C++11, static storage.
class Logger {
public:
```

```
    static Logger& instance() {
        static Logger inst;           // constructed once, on first call
        return inst;
    }
    void write(const char* msg) { /* UART/ITM/etc. */ }
private:
    Logger() { /* init UART pinmux etc. */ }
    ~Logger() = default;
    Logger(const Logger&) = delete;
    Logger& operator=(const Logger&) = delete;
};

// Usage styles:
Logger::instance().write("booting...");
auto& log = Logger::instance();  // 'auto&' reference
log.write("ok");
```

Embedded C++ notes:

- ✅ Good fit for truly unique HW (e.g., watchdog, single UART console).
- ⚠️ Harder to unit-test; consider dependency injection instead (pass references).
- ⚠️ Avoid dynamic allocation; prefer function-local static as above (deterministic construction the first time you call it).

Singleton is a creational design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance.

Applicability:

Use the Singleton pattern when a class in your program should have just a single instance available to all clients; for example, a single database object shared by different parts of the program.

- The Singleton pattern disables all other means of creating objects of a class except for the special creation method. This method either creates a new object or returns an existing one if it has already been created.

Use the Singleton pattern when you need stricter control over global variables.

- Unlike global variables, the Singleton pattern guarantees that there's just one instance of a class. Nothing, except for the Singleton class itself, can replace the cached instance.

- Note that you can always adjust this limitation and allow creating any number of Singleton instances. The only piece of code that needs changing is the body of the getInstance method.

Structural Patterns

Explain how to assemble objects and classes into larger structures, while keeping these structures flexible and efficient.

Adapter

What it does: Translates one interface to another so existing code can use a component it wasn't designed for. Lets you swap vendors/MCUs without changing higher-level logic.

```
// Target (what the app expects)
struct ISpi {
    virtual ~ISpi() = default;
    virtual void xfer(const uint8_t* tx, uint8_t* rx, size_t n) = 0;
};

// Adaptee (vendor HAL)
extern "C" {
    void SPI1_WriteRead(const uint8_t* tx, uint8_t* rx, size_t n); // HAL
}

// Adapter: wraps vendor HAL to ISpi
class Spi1Adapter : public ISpi {
public:
    void xfer(const uint8_t* tx, uint8_t* rx, size_t n) override {
        SPI1_WriteRead(tx, rx, n);
    }
};

// Usage (multiple declarations)
Spi1Adapter spi1;
```

```
ISpi& bus = spi1;                // reference
ISpi* pbus = &spi1;               // pointer
bus.xfer(tx_buf, rx_buf, len);
```

Embedded C++ notes:

- ✅ Clean portability: keep app logic independent of register/HAL details.
- ✅ Enables mocking in tests (fake ISpi).
- If you need zero overhead, use templated adapters (static polymorphism) instead of virtuals.

Adapter design pattern is a structural design pattern used to allow classes with incompatible interfaces to work together. The Adapter pattern involves creating an adapter class that wraps an existing class (or module) and provides a new interface that the client expects.

Applicability:

Use the Adapter class when you want to use some existing class, but its interface isn't compatible with the rest of your code.

- The Adapter pattern lets you create a middle-layer class that serves as a translator between your code and a legacy class, a 3rd-party class or any other class with a weird interface.

Use the pattern when you want to reuse several existing subclasses that lack some common functionality that can't be added to the superclass.

- Put the missing functionality into an adapter class. Then you would wrap objects with missing features inside the adapter, gaining needed features dynamically. For this to work, the target classes must have a common interface, and the adapter's field should follow that interface.

Behavioral Patterns

Take care of effective communication and the assignment of responsibilities between objects.

Command

What it does: Encapsulates a request as an object: you can queue, log, undo, or defer work. Great to move actions out of ISRs and into a worker/thread.

```cpp
// Command interface
struct ICommand { virtual ~ICommand() = default; virtual void exec() = 0; };

// Concrete commands
class EraseFlash : public ICommand {
    uint32_t addr_, len_;
public:
    EraseFlash(uint32_t a, uint32_t l) : addr_(a), len_(l) {}
    void exec() override { flash_erase(addr_, len_); }
};

class WriteFlash : public ICommand {
    uint32_t addr_; const uint8_t* data_; size_t len_;
public:
    WriteFlash(uint32_t a, const uint8_t* d, size_t l) : addr_(a), data_(d), len_(l) {}
    void exec() override { flash_write(addr_, data_, len_); }
};

// Queue (heapless ring)
template<size_t N>
struct CmdQueue {
    ICommand* q[N]{}; size_t h=0,t=0;
    bool push(ICommand* c){ size_t n=(h+1)%N; if(n==t) return false; q[h]=c; h=n; return true; }
    ICommand* pop(){ if(t==h) return nullptr; auto* c=q[t]; t=(t+1)%N; return c; }
};

CmdQueue<8> cmdq;

// Producer (e.g., setup or from an ISR-safe stub)
EraseFlash erase{0x08020000, 4096};
WriteFlash write{0x08021000, img, img_len};
cmdq.push(&erase);
cmdq.push(&write);

// Consumer (worker task / main loop)
void worker(){
    if (auto* c = cmdq.pop()) c->exec();
```

```
}
```

Alternate declarations:

- auto* pc = &erase; cmdq.push(pc);
- Use placement new into a static pool to avoid heap.

Embedded C++ notes:

- ✅ Perfect for defer work from ISR → queue → worker thread.
- ✅ Enables scripted sequences (bootloader, calibration).
- ⚠️ Prefer static storage (no new/delete), or a fixed pool with custom deleter.

The command pattern is a behavioral design pattern used to capture a function call together with required arguments – allowing us to execute those functions with a delay.

Applicability:

Use the Command pattern when you want to parameterize objects with operations.

- The Command pattern can turn a specific method call into a stand-alone object. This change opens up a lot of interesting uses: you can pass commands as method arguments, store them inside other objects, switch linked commands at runtime, etc.

Use the Command pattern when you want to queue operations, schedule their execution, or execute them remotely.

- As with any other object, a command can be serialized, which means converting it to a string that can be easily written to a file or a database. Later, the string can be restored as the initial command object. Thus, you can delay and schedule command execution. But there's even more! In the same way, you can queue, log or send commands over the network.

Use the Command pattern when you want to implement reversible operations.

- To be able to revert operations, you need to implement the history of performed operations. The command history is a stack that contains all executed command objects along with related backups of the application's state.

Observer

What it does: Implements publish/subscribe: subjects notify many observers about events without hard coupling.

```cpp
#include <array>
using Callback = void(*)(int);

class ButtonSubject {
public:
    template<size_t N>
    void set_storage(std::array<Callback,N>& store){ cbs_=store.data(); cap_=N; }

    bool subscribe(Callback cb){
        for(size_t i=0;i<cap_;++i) if(!cbs_[i]){ cbs_[i]=cb; return true; }
        return false;
    }

    void notify(int event){
        for(size_t i=0;i<cap_;++i) if(cbs_[i]) cbs_[i](event);
    }
private:
    Callback* cbs_{};
    size_t cap_{};
};

void on_press(int e){ /* handle */ }
void on_log(int e){ /* log */ }

std::array<Callback,4> slots{};
ButtonSubject btn;
btn.set_storage(slots);              // no heap
btn.subscribe(&on_press);
btn.subscribe(&on_log);

// Somewhere (e.g., debounced edge)
btn.notify(/*event=*/1);
```

Variants:

- Use std::function<void(int)> if you have C++17 and can afford it.
- Use templated observers for compile-time binding (zero overhead).

Embedded C++ notes:

- ✅ Decouples ISR producers from multiple consumers (UI, logger, control).
- ✅ Use ring buffer between ISR and subject to keep ISR short.
- ⚠️ Avoid dynamic allocation; pre-register observers.

This pattern is particularly useful when multiple components need to react to changes in data from a central source. The Observer pattern is often used in event-driven systems to publish events to subscribed objects, usually by calling a method on them. An object that publishes events is called a subject or publisher. Objects that receive events from a publisher are called observers or subscribers

Applicability:

Use the Observer pattern when changes to the state of one object may require changing other objects, and the actual set of objects is unknown beforehand or changes dynamically.

- The Observer pattern lets any object that implements the subscriber interface subscribe for event notifications in publisher objects.

Use the pattern when some objects in your app must observe others, but only for a limited time or in specific cases.

- The subscription list is dynamic, so subscribers can join or leave the list whenever they need to.

State

What it does: Models behavior as finite states with transitions. Replaces giant if/switch ladders with clear structure. Two common forms:

1. Class-based State objects (polymorphic).
2. Table-driven / enum + function pointers (zero-overhead).

Class-based:

```
struct Context;       // forward

struct IState {
    virtual ~IState() = default;
    virtual void on_enter(Context&) {}
    virtual void on_tick(Context&) = 0;
```

```
};

struct Context {
    IState* state{};
    bool power_ok=false, sensor_ready=false;

    void set(IState& s){ state = &s; state->on_enter(*this); }
    void tick(){ state->on_tick(*this); }
};

struct Booting : IState {
    void on_tick(Context& c) override {
        if (c.power_ok) c.set(WaitingSensor::instance());
    }
    static Booting& instance(){ static Booting s; return s; }
};

struct WaitingSensor : IState {
    void on_tick(Context& c) override {
        if (c.sensor_ready) c.set(Running::instance());
    }
    static WaitingSensor& instance(){ static WaitingSensor s; return s; }
};

struct Running : IState {
    void on_tick(Context&) override { /* control loop */ }
    static Running& instance(){ static Running s; return s; }
};

// Wire-up
Context ctx;
ctx.set(Booting::instance());
ctx.power_ok = true; ctx.tick();        // -> WaitingSensor
ctx.sensor_ready = true; ctx.tick();    // -> Running
```

Enum/table:

```
enum class S{ Boot, Wait, Run };
S st = S::Boot;

void tick(bool power_ok, bool sensor_ready){
   switch(st){
      case S::Boot:  if(power_ok) st=S::Wait; break;
      case S::Wait:  if(sensor_ready) st=S::Run; break;
      case S::Run:   /* loop */ break;
   }
}
```

Embedded C++ notes:

- ✅ Great for boot sequences, comms protocols, power modes.
- Use table-driven or templates for zero-virtual-cost FSMs.
- Keep transition actions short; heavy work in the main loop/task.

This pattern is “state-centric,” meaning each state is encapsulated as its own class.

Applicability:

Use the State pattern when you have an object that behaves differently depending on its current state, the number of states is enormous, and the state-specific code changes frequently.

- The pattern suggests that you extract all state-specific code into a set of distinct classes. As a result, you can add new states or change existing ones independently of each other, reducing the maintenance cost.

Use the pattern when you have a class polluted with massive conditionals that alter how the class behaves according to the current values of the class’s fields.

- The State pattern lets you extract branches of these conditionals into methods of corresponding state classes. While doing so, you can also clean temporary fields and helper methods involved in state-specific code out of your main class.

Use State when you have a lot of duplicate code across similar states and transitions of a condition-based state machine.

- The State pattern lets you compose hierarchies of state classes and reduce duplication by extracting common code into abstract base classes.

Common Q&A

1. What is exception in C++?

Runtime abnormal conditions that occur in the program are called exceptions.

These are of 2 types:

- Synchronous
- Asynchronous

C++ has 3 specific keywords for handling these exceptions: – try – catch – throw

---

2. What is stack in C++?

A linear data structure which implements all the operations (push, pop) in LIFO (Last In First Out) order. Stack can be implemented using either arrays or linked list.

The operations in Stack are:

- Push: adding element to stack.
- Pop: removing element from stack.
- isEmpty: returns true if stack is empty.
- Top: returns the top most element in stack.

---

3. Define token in C++

A token is the smallest unit of code that the compiler recognizes during compilation.

- Identifiers – Names of variables, functions, classes, etc.
- Keywords – Reserved words like if, int, while.
- Literals – Constant values like numbers (42) or strings ("text").
- Operators – Symbols like +, -, *, &&.
- Comments – Notes in code (//, /* */) that are ignored by the compiler.
- Whitespace – Spaces, tabs, and newlines that separate tokens but aren’t processed.

---

4. Purpose of the “delete” operator in C++?

The delete operator in C++ frees memory allocated by new and calls the destructor to clean up the object.

---

5. Difference between new and malloc() in C++?

- new is an operator that allocates memory and also calls the constructor to initialize objects. It returns a pointer of the appropriate type and throws an exception if allocation fails.
- malloc() is a C library function that only allocates raw memory but does not call constructors. It returns a void* that needs to be cast to the appropriate type and returns nullptr (or NULL) if allocation fails.

---

6. Difference between struct and class in C++?

By default, members of a struct are public, meaning they can be accessed from outside the struct. In contrast, members of a class are private by default, restricting access only within the class itself.

---

7. Difference between ++i and i++?

++i (pre-increment) increments the value first and then returns it, while i++ (post-increment) returns the value first and then increments it.

---

8. Difference between a shallow copy and a deep copy?

- A shallow copy copies the values of the members of an object, but it does not clone dynamically allocated memory or pointed-to objects.
- A deep copy duplicates everything, including dynamically allocated memory, ensuring that the copy is independent of the original.

---

9. Use of the explicit keyword in C++?

The explicit keyword is used to mark constructors or conversion operators to prevent implicit conversions. It ensures that a constructor or operator is not used for automatic conversions unless explicitly invoked by the programmer.

---

10. What is the ‘this’ pointer in C++?

The ‘this’ pointer is an implicit pointer passed to all non-static member functions of a class. It points to the current instance of the class, allowing access to the object’s members and methods.

11. What is the static keyword used for in C++?

The static keyword can be used in several contexts:

- For variables inside functions, it preserves the variable's value between function calls.
- For class members, it means the member is shared across all instances of the class (i.e., it is not tied to any specific object).
- For functions, it limits the scope of the function to the file it is declared in.

---

12. Explain the concept of object slicing in C++.

Object slicing occurs when an object of a derived class is assigned to an object of a base class, and the derived class-specific data is "sliced off". This happens because the base class does not know about the additional members of the derived class, resulting in loss of information.

---

13. Difference between const and constexpr in C++?

const means that the value of a variable cannot be changed after initialization, but it may be computed at runtime. constexpr indicates that the value of a variable is constant and must be computed at compile-time.

---

14. Significance of inline functions in C++?

The inline keyword suggests to the compiler to replace the function call with the function's body during compilation, potentially improving performance by eliminating function call overhead, especially for small functions. However, the compiler can choose to ignore the inline suggestion.

---

15. What is the std::move function in C++?

std::move is used to enable moving of resources from one object to another. It casts an object to an rvalue reference, allowing resources to be transferred without copying, improving performance, especially with large objects.

Move constructors and move assignment operators are special functions that allow resources to be transferred from one object to another, instead of copying. This is useful for optimizing performance by avoiding deep copies. Declared using rvalue references (&&).

---

16. Difference between static_cast, dynamic_cast, const_cast, and reinterpret_cast?

Type casting converts one data type into another. C++ provides four types of casts:

- static_cast: Used for normal type conversions (e.g., converting between related classes or basic types). Compile-time type checking.
- dynamic_cast: Used for safe downcasting in polymorphic class hierarchies.
- const_cast: Used to add or remove const qualifier from a variable.
- reinterpret_cast: Used for low-level, potentially unsafe type conversions between different types.

---

17. What is an rvalue reference in C++?

An rvalue reference is a reference to a temporary object (rvalue) that can be used to implement move semantics. It is declared using && (double ampersand), allowing efficient transfer of resources from one object to another.

---

18. What are friend functions in C++?

A friend function is a function that can access the private and protected members of a class. It is not a member function but is granted access through a friend declaration inside the class.

---

19. What is the nullptr keyword in C++?

nullptr is a type-safe null pointer constant introduced in C++11. It is used to indicate that a pointer does not point to any object or valid memory.

---

20. Explain the concept of std::tuple in C++.

std::tuple is a fixed-size collection of elements of potentially different types. It allows grouping heterogeneous values together and is similar to a structure, but more flexible as it can hold any type in any order.

---

21. Purpose of the mutable keyword in C++?

The mutable keyword allows modification of a member variable even in a const object or const member function. It is often used when caching or lazy initialization is needed.

22. Difference between std::list and std::vector?

- std::vector: Provides fast random access and is better for scenarios where elements are frequently accessed by index. However, insertions and deletions in the middle are slow.
- std::list: Implements a doubly linked list, making insertions and deletions fast, but random access is not possible (only sequential traversal).

---

23. Advantages of C++ over C?

C++ provides features like object-oriented programming, type safety with stronger typing, templates for generic programming, and a standard library that includes containers, algorithms, and iterators, making it more versatile and powerful than C.

---

24. Difference between override and final in C++11.

- override: Indicates that a member function is meant to override a base class function, ensuring at compile-time that the function signature matches.
- final: Prevents further overriding of a virtual function in derived classes or inheritance from a class.

---

25. Significance of std::string_view in C++17?

std::string_view is a lightweight, non-owning reference to a string. It allows efficient manipulation of strings without creating copies, making it faster for read-only string operations.

---

26. Difference between throw(), noexcept, and noexcept(true)?

- throw(): Deprecated in C++11, it indicated that a function does not throw exceptions.
- noexcept: Introduced in C++11, it specifies that a function is guaranteed not to throw exceptions.
- noexcept(true): Explicitly indicates that the function does not throw exceptions.

---

27. Difference between delete and delete[]?

- delete: Used to deallocate memory for a single object allocated with new.
- delete[]: Used to deallocate memory for arrays allocated with new[].

---

28. What is a function pointer in C++?

A function pointer is a pointer that stores the address of a function. It allows functions to be passed as arguments to other functions or stored in containers.
Example:

```
void foo() { std::cout << "Hello"; }
void (*funcPtr)() = foo;
funcPtr();
```

---

29. What is a dangling pointer in C++?

A dangling pointer is a pointer that points to memory that has been deallocated or released. Accessing it leads to undefined behavior.

---

30. What is std::atomic in C++?

std::atomic provides a way to perform lock-free and thread-safe operations on shared variables. It is commonly used in multi-threaded programs.

---

31. What are variadic templates in C++?

Variadic templates allow a function or class template to accept an arbitrary number of template arguments. Defined using ... (ellipsis).
Example:

```
template<typename... Args>
void func(Args... args) { }
```

---

32. Differences between inline and constexpr functions?

- inline: Suggests to the compiler to replace function calls with the function body to reduce overhead.

- constexpr: Ensures that the function can be evaluated at compile-time if the input is constant.

---

33. Difference between typedef and using in C++?

Both are used for type aliasing. using is introduced in C++11 and is more readable, especially for template aliases.
Example:

```
typedef int* IntPtr;  // Typedef
using IntPtr = int*;  // Using
```

---

34. What is the rule of three in C++?

If a class requires a user-defined destructor, copy constructor, or copy assignment operator, it likely needs all three. This is because the class manages resources that need explicit handling.

---

35. What are the major differences between C++11 and C++17?

- C++11: Introduced features like auto, lambda expressions, nullptr, smart pointers, and move semantics.
- C++17: Added std::optional, std::variant, std::string_view, structured bindings, and constexpr improvements.

References

Book: *C++ in Embedded Systems, A practical transition from C to modern C++* by Amar Mahmutbegović, 1st Ed, 2025.

https://medium.com/@oleksandra_shershen/solid-principles-implementation-and-examples-in-c-99f0d7e3e868

https://refactoring.guru/design-patterns/cpp